RESTAURANT ORDERING AND KITCHEN DISPLAY SYSTEM

A PROJECT REPORT

Submitted by

ROSHAN BRIGHT R
(2116240701439)

in partial fulfilment of the End Semester Examination for the course

CB23F35 -
LOW-CODE APPLICATION DEVELOPMENT USING ZOHO CREATOR

in

BACHELOR OF
COMPUTER SCIENCE AND ENGINEERING



RAJALAKSHMI ENGINEERING COLLEGE
(An Autonomous Institution, Affiliated to Anna University, Chennai)
Rajalakshmi Nagar, Thandalam, Tamil Nadu - 602 105

NOVEMBER 2026

2407439

RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

BONAFIDE CERTIFICATE

Certified that this project report titled "Restaurant Ordering and Kitchen Display System" is the bonafide work of ROSHAN BRIGHT R (2116240701439), who carried out the individual project work on the Zoho Creator low-code platform for the course CB23F35 - Low-Code Application Development using Zoho Creator, during the academic year [2026-2027], Semester [V], under my supervision.

2407 0143 9

SIGNATURE

Dr MANORANJINI J
HEAD OF THE DEPARTMENT
Department of Computer Science and Engineering
Rajalakshmi Engineering College
Thandalam - 602 105

SIGNATURE

Dr.Jinu Sophia
COURSE FACULTY
Academic Head
Department of Computer Science and Engineering
Rajalakshmi Engineering College

Submitted for the End Semester Examination (Individual Project) held on 11/11/2026.

INTERNAL EXAMINER

EXTERNAL EXAMINER

DECLARATION

I, ROSHAN BRIGHT R ([2116240701439]), hereby declare that the project report titled "Restaurant Ordering and Kitchen Display System", submitted for the End Semester Examination of the course CB23F35 - Low-Code Application Development using Zoho Creator, is a record of original work carried out by me individually. The Zoho Creator application described in this report, including its data model, forms, reports, pages, Deluge scripts and workflows, was designed and developed by me.

I further declare that this work has not been submitted, in part or in full, for the award of any other degree or course credit. Every template, sample application, AI-assisted feature (such as Zia or the AI application builder), code snippet or external resource used in this work has been duly acknowledged in the report. An AI assistant was used for guidance on the design, scripting and documentation of this project, and all scripts were run and verified by me on the platform.

Place: Chennai
Date: 11-11-2026

SIGNATURE OF THE STUDENT
ROSHAN BRIGHT R
[2116240701439]

2407014 39

ACKNOWLEDGEMENT

I express my sincere gratitude to our Chairman Thiru. S. Meganathan, Chairperson Dr. (Mrs.) Thangam Meganathan and Vice-Chairman Mr. M. Abhay Shankar for providing the infrastructure and facilities that made this work possible.

I am grateful to our Principal, Dr. S.N. Murugesan, Rajalakshmi Engineering College, for the constant encouragement extended to students to take up industry-relevant learning.

I thank MANORANJINI J, Head of the Department, Department of Computer Science And Engineering, for the guidance and support provided throughout the course.

I owe my sincere thanks to my course faculty, DR.JINU SOPHIA, ACADEMIC HEAD, for the valuable guidance, reviews and feedback that shaped this application and report.

I also acknowledge Zoho Corporation for providing access to the Zoho Creator platform and learning resources, and I thank my family and friends for their unwavering support.

ROSHAN BRIGHT R

2407014 39

ABSTRACT

Small and medium restaurants still run most of their daily work on paper order slips, verbal instructions to the kitchen, a handwritten stock register and a cash book. This causes lost or misread orders, no shared view of order status, ingredient shortages that are noticed only when a dish cannot be served, and sales totals that must be added by hand. The people affected are waiters, kitchen staff, cashiers and the restaurant manager.

This project presents a Restaurant Ordering and Kitchen Display System built on the Zoho Creator low-code platform. Low-code suits the problem because it is a forms-over-data application with status-driven rules, which can be assembled through configuration with only short scripts for business logic. The application has nine linked forms (Category, Menu_Item, Ingredient, Recipe, Restaurant_Table, Order, Order_Item, Payment and Purchase_Request), nine list reports, a Kanban Kitchen Display, four pivot charts and a Sales Dashboard page with KPI tiles. Ten form workflows written in Deluge fetch prices, calculate line and order totals, mark tables Occupied or Free, block unavailable dishes, deduct ingredient stock from recipes exactly once, raise purchase requests when stock is low, and complete the order on payment. Because the free plan allows only a demo of subforms, the order lines are held in a separate Order_Item form linked by a lookup.

The application was tested with sample data covering seventeen test cases, and nine defects found during development were fixed and logged. The work shows that the full order-to-payment cycle can be automated without conventional programming, removes manual price and total calculation, gives the kitchen a live status board, keeps stock current automatically and provides instant sales analytics. Limits of the free plan, such as the absence of public links, scheduled e-mail and Blueprint, are documented together with the workarounds used and the planned enhancements.

Keywords: *Low-Code Development; Zoho Creator; Deluge Scripting; Workflow Automation; Inventory Management; Restaurant Management*

2407439

PROJECT AT A GLANCE

Attribute	Details
Project Title	Restaurant Ordering and Kitchen Display System
Application Domain	Hospitality / Restaurant Management
Student Name & Register No.	ROSHAN BRIGHT R - [2116240701439]
Zoho Creator Edition	Free (trial period completed)
Application Name (Link Name)	Restaurant Ordering System (restaurant-ordering-system)
Published / Live URL	https://creatorapp.zoho.in/roshanbright.r.2024.cse_r/restaurant-ordering-system/
Demo Credentials (per role)	Administrator
Number of Forms / Subforms	9 / 0 (subform replaced by Order_Item form with lookup)
Number of Reports (by type)	List: 9; Kanban: 1; Calendar: 0; Pivot/Chart: 4; Others: 0
Number of Pages / Dashboards	2 (Sales Dashboard with KPI tiles and four charts; one default Dashboard page)
Number of Workflows	Form: 10; Schedule: 0; Approval: 0; Blueprint: 0; Batch: 0
Deluge Functions / Custom Actions	0 standalone functions; 10 workflow scripts plus 1 dashboard script block
Integrations	None in this version (planned: Zoho Mail, payment gateway)
Roles & Profiles Defined	Default profiles (Administrator, Write, Read, Developer, Customer) and role CEO; design matrix for Waiter, Kitchen, Cashier
Environments Used	Single environment (free plan); .ds exports as backups
Date of Production Deployment	11-11-2026

2407101439

TABLE OF CONTENTS

ABSTRACT ..... **iv**
PROJECT AT A GLANCE ..... **v**
LIST OF TABLES ..... **viii**
LIST OF FIGURES ..... **x**
LIST OF ABBREVIATIONS ..... **xi**
INTRODUCTION **1**
1.1 Background 1
1.2 Problem Statement 1
1.3 Objectives 2
1.4 Scope of the Project 2
1.5 Justification for a Low-Code Approach 2
1.6 Organisation of the Report 3
LITERATURE SURVEY AND SYSTEM STUDY **4**
2.1 Study of the Existing System 4
2.2 Limitations of the Existing System 4
2.3 Low-Code versus Traditional Development 4
2.4 Literature Survey 5
2.5 Emerging Trends in Low-Code Development 6
2.6 Proposed System 6
REQUIREMENT ANALYSIS **8**
3.1 Stakeholders and User Roles 8
3.2 Functional Requirements 8
3.3 Non-Functional Requirements 9
3.4 Hardware and Software Requirements 9
3.5 Use Case Model 10
SYSTEM DESIGN **12**
4.1 System Architecture 12
4.2 Data Model 12
4.3 Form Design 14
4.4 Process and Workflow Design 16
4.5 User Interface and Navigation Design 17
4.6 Access Control Design 18
IMPLEMENTATION **19**
5.1 Application Setup 19
5.2 Forms and Data Capture 19
5.3 Reports 20
5.4 Pages and Dashboards 21
5.5 Deluge Scripting 22
5.6 Workflows and Automation 24
5.7 Integrations 25

2407439

5.8 User Management and Role-Based Access 26
5.9 AI and Analytics Features 26
5.10 Pro-Code Extension 27
5.11 Mobile and Portal Access 27
TESTING AND DEBUGGING 28
6.1 Testing Strategy 28
6.2 Test Cases 28
6.3 Debugging Deluge Scripts 29
6.4 Defect Log 30
6.5 User Acceptance Testing 31
DEPLOYMENT, SECURITY AND GOVERNANCE 32
7.1 Environments and Deployment Strategy 32
7.2 Performance Optimisation 32
7.3 Security Features 33
7.4 Governance and Administration 33
RESULTS AND DISCUSSION 35
8.1 Application Output 35
8.2 Achievement of Objectives 36
8.3 Comparison with the Existing System 36
8.4 Limitations of the Low-Code Approach 37
8.5 Reflection on the No-Code, Low-Code and Pro-Code Spectrum 37
CONCLUSION AND FUTURE ENHANCEMENTS 39
9.1 Conclusion 39
9.2 Future Enhancements 39
9.3 Learning Outcomes and Career Relevance 39
REFERENCES 40
APPENDIX A - ZOHO CREATOR FEATURE COVERAGE CHECKLIST 41
APPENDIX B - COURSE OUTCOME MAPPING 43
APPENDIX C - COMPLETE DELUGE SOURCE CODE 44
APPENDIX D - USER MANUAL 47
D.1 Accessing the Application 47
D.2 Instructions for the Waiter 47
D.3 Instructions for the Kitchen Staff 47
D.4 Instructions for the Cashier 47
D.5 Instructions for the Manager 47
APPENDIX E - EVALUATION RUBRIC 49
APPENDIX F - VIVA-VOCE RECORD SHEET 51

240701439

LIST OF TABLES

Table 2.1: Comparison of Low-Code and Traditional Development for the Proposed Application 5
Table 2.2: Summary of Literature Survey 5
Table 3.1: Stakeholders and User Roles 8
Table 3.2: Functional Requirements 8
Table 3.3: Non-Functional Requirements 9
Table 3.4: Hardware and Software Requirements 9
Table 3.5: Use Case Description: Place an Order and Add Dishes 10
Table 3.6: Use Case Description: Accept an Order and Complete Payment 11
Table 4.1: Entities and Relationships 13
Table 4.2: Field Specification: Category Form 14
Table 4.3: Field Specification: Ingredient Form 14
Table 4.4: Field Specification: Menu_Item Form 14
Table 4.5: Field Specification: Restaurant_Table Form 15
Table 4.6: Field Specification: Recipe Form 15
Table 4.7: Field Specification: Order Form 15
Table 4.8: Field Specification: Order_Item Form 16
Table 4.9: Field Specification: Payment Form 16
Table 4.10: Field Specification: Purchase_Request Form 16
Table 4.11: Role-Permission Matrix 18
Table 5.1: Reports Implemented 20
Table 5.2: Deluge Scripts Implemented 22
Table 5.3: Workflows Implemented 24
Table 5.4: Integrations Implemented 25
Table 6.1: Test Cases 28
Table 6.2: Defect Log 30
Table 6.3: User Acceptance Feedback 31
Table 7.1: Deployment Checklist 32
Table 7.2: Performance Optimisation Results 33
Table 7.3: Security Controls Implemented 33
Table 8.1: Mapping of Objectives to Outcomes 36
Table 8.2: Existing System versus Proposed Application 36
Table 8.3: Position of Application Components on the Spectrum 37

2407439

Table A.1: Feature Coverage Checklist 41
Table B.1: Mapping of Course Outcomes to Report Chapters 43
Table E.1: Evaluation Rubric for the Individual Project 49
Table E.2: Performance Level Descriptors 49
Table F.1: Viva-Voce Questions and Responses 51
Table F.2: Marks Summary 51

2407014 39

LIST OF FIGURES

Figure 2.1: As-Is Process Flow of the Existing System 4
Figure 3.1: Use Case Diagram of the Application 10
Figure 4.1: System Architecture of the Application 12
Figure 4.2: Entity-Relationship Diagram 13
Figure 4.3: Workflow / Process Flow for the Order Lifecycle 17
Figure 4.4: Navigation Structure and Wireframe of the Sales Dashboard 18
Figure 5.1: Application Home and Navigation Sections 19
Figure 5.2: Menu_Item Form in the Form Builder 20
Figure 5.3: Order_Item Form in Live Mode 20
Figure 5.4: All Orders Report 21
Figure 5.5: Sales Dashboard Page 22
Figure 5.6: Workflow Configuration (Mark Table Occupied, Add New Action) 25
Figure 5.7: Roles and Permission Profiles Configuration 26
Figure 5.8: Pivot Chart Builder for Sales by Category 27
Figure 6.1: Deluge Debug Output: Variable 'row' Is Not Defined 30
Figure 7.1: Administration Settings 34
Figure 7.2: Delete Field Dependency Check 34
Figure 8.1: Scenario Step 1: Order Placed for a Table 35
Figure 8.2: Scenario Step 2: Availability Validation on an Order Item 35
Figure 8.3: Scenario Step 3: Sales Dashboard After Paid Orders 36
Figure 8.4: Subform Restriction on the Free Plan 37

2407439

LIST OF ABBREVIATIONS

Abbreviation	Expansion
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
CSV	Comma-Separated Values
DS	Deluge Script (script ID prefix) / Zoho Creator application definition file (.ds)
ER	Entity-Relationship
FR	Functional Requirement
JSON	JavaScript Object Notation
KPI	Key Performance Indicator
LCAP	Low-Code Application Platform
NFR	Non-Functional Requirement
OAuth	Open Authorization
RBAC	Role-Based Access Control
REST	Representational State Transfer
SDLC	Software Development Life Cycle
UAT	User Acceptance Testing
UI	User Interface
Zia	Zoho's AI assistant

2407439

CHAPTER 1
INTRODUCTION

1.1 Background

Restaurants are among the most process-intensive small businesses. A single dining visit passes through table allocation, order taking, kitchen preparation, stock consumption, billing and payment, and each of these steps is usually handled by a different person. In many small and medium restaurants the whole chain still runs on paper order slips, verbal instructions, a handwritten stock register and a cash book.

This project considers a mid-sized dine-in restaurant with a menu of around ten dishes grouped into starters, main course and beverages, a floor of five tables, and four kinds of staff: waiters who take orders, kitchen staff who prepare them, a cashier who collects payment, and a manager who owns the menu, the inventory and the sales figures. The business process addressed is the complete order-to-payment cycle: from the moment a table is occupied to the moment the bill is paid and the table is free again, including the consumption of ingredients along the way.

1.2 Problem Statement

Restaurant operations that depend on paper slips and manual registers suffer from lost or misread orders, no shared view of what the kitchen is preparing, stock that is discovered to be empty only when a dish cannot be served, and sales totals that must be added up by hand. The problem is to provide a single, role-aware, web-based application that records every order against a table, tracks it through the kitchen, deducts ingredient stock automatically and presents sales analytics, without the cost and delay of a conventionally coded system.

- Order slips are handwritten, so items are lost, duplicated or misread between the waiter and the kitchen.
- The kitchen has no live view of order status, and waiters must walk to the kitchen to find out whether a dish is ready.
- Ingredient stock is updated manually, so shortages are noticed late and re-ordering is reactive.
- Bills and daily sales are calculated by hand, which is slow and error-prone, and there is no quick view of the best-selling dishes.

2407143

- Table availability is tracked informally, so tables are not released promptly after payment.

1.3 Objectives

- To design a normalised data model and forms for menu, recipes, ingredients, tables, orders, order items, payments and purchase requests.
- To automate price lookup, line and order totals, table status, stock deduction and payment completion using Deluge scripting and form workflows.
- To provide a kitchen display (Kanban report) and a sales dashboard with charts for the restaurant manager.
- To prevent invalid transactions, such as ordering an unavailable dish or deducting stock twice, through validation and flag fields.
- To deliver the application on Zoho Creator with governance controls suited to the platform edition available, and to document the limits met on the free plan.

1.4 Scope of the Project

In scope: master data (categories, menu items, ingredients, recipes, tables); order capture for a table; adding dishes to an order with automatic pricing; kitchen status tracking through Placed, Accepted, Preparing, Ready, Served, Billed and Paid; automatic ingredient deduction and purchase requests; payment recording by Cash, Card or UPI; list, Kanban and pivot-chart reports; and a Sales Dashboard page. The application runs on the web through the Zoho Creator live app.

Out of scope: a public, QR-code-driven customer ordering page (public permalinks are not available on the free plan), online payment gateways, scheduled e-mail reports, multi-branch support, and a native mobile build. These are listed as future enhancements in Section 9.2.

1.5 Justification for a Low-Code Approach

A restaurant ordering system is a classic forms-over-data application: it is made of data entry forms, relationships between records, status-driven workflows and summary reports. Low-code platforms are designed for exactly this class of problem, where much of the application can be assembled by configuration and only the business rules need code [1], [2]. Zoho Creator supplies hosting, a managed database, a form builder, report and page builders, role-based access and the Deluge scripting language in one

2407439

environment, so no web server, database or front-end framework had to be set up [3], [4].

For this project the advantages were concrete. Nine forms with lookup relationships were created in a few hours through the form builder; the ten automations are short Deluge scripts of two to twenty lines; and the Kitchen Display and the dashboard charts were produced by configuring reports rather than programming user interfaces. The result can be maintained by a citizen developer, such as the restaurant manager, which is a major business benefit of the low-code approach [2].

1.6 Organisation of the Report

Chapter 2 reviews the existing system and related low-code literature. Chapter 3 presents the requirement analysis. Chapter 4 describes the system design. Chapter 5 details the implementation on Zoho Creator. Chapter 6 covers testing and debugging. Chapter 7 discusses deployment, performance optimisation, security and governance. Chapter 8 presents results and discussion, and Chapter 9 concludes the report with future enhancements.

24070139

CHAPTER 2
LITERATURE SURVEY AND SYSTEM STUDY

2.1 Study of the Existing System

In the existing manual system a customer is seated and the waiter writes the order on a paper slip. The slip is carried to the kitchen, where the cook prepares the dishes without any visible status. Ingredients are consumed from the store and the stock register is updated by hand, usually at the end of the day. When the meal is over the waiter writes the bill, the cashier collects the amount and the sale is entered in a ledger. If a slip is lost or unclear, the waiter has to return to the table and the order is taken again.

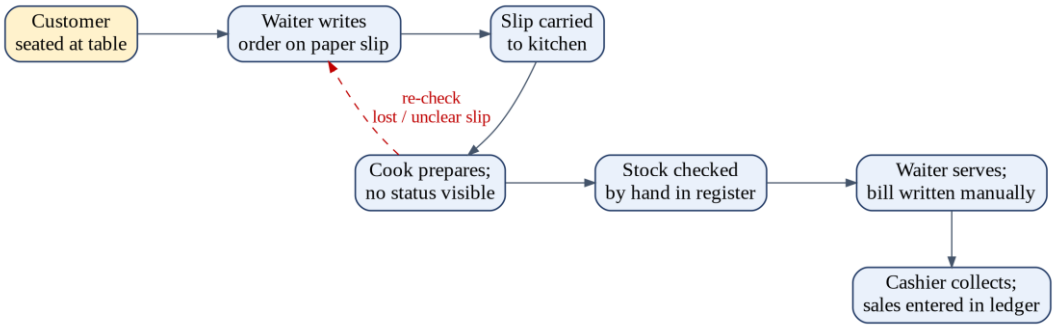


Figure 2.1: As-Is Process Flow of the Existing System

2.2 Limitations of the Existing System

- Orders depend on handwriting and physical movement of slips, so errors and delays are common.
- No real-time status: neither the waiter nor the customer can see whether an order is accepted, being prepared or ready.
- Stock is reconciled manually, so ingredient shortages and wastage are not visible until it is too late.
- Billing and sales summaries are calculated manually and cannot be analysed by dish, category or time.
- There is no access control: anyone with the register can change prices, stock or totals.

2.3 Low-Code versus Traditional Development

Table 2.1 compares the two approaches for this particular application. The figures are the author's indicative estimates for a single developer working part-time, not measured benchmarks.

Table 2.1: Comparison of Low-Code and Traditional Development for the Proposed Application

Criterion	Traditional (Pro-Code) Development	Low-Code on Zoho Creator
Development time	About 6 to 8 weeks: schema design, back-end APIs, front-end screens, testing and deployment	About 2 weeks: forms, workflows, reports and dashboard configured and scripted
Skills required	Web framework, REST API design, SQL, front-end development, DevOps	Data modelling, form and report configuration, basic Deluge scripting
Hosting & infrastructure	Server or cloud instance, database, SSL and backups to be arranged and maintained	Provided by Zoho Creator cloud; no infrastructure to manage
Mobile application	Separate responsive design or native app needed	Forms and reports are responsive; Zoho Creator mobile app available
Security & access control	Authentication, sessions and permissions coded by the developer	Built-in user management, profiles and permissions
Maintenance & change	Code changes, rebuild and redeploy for every change	Edit form or script in the builder and save; changes are live
Customisation limits	Practically unlimited	Bounded by platform features and plan; pro-code extensions needed beyond them
Cost	Developer time plus hosting costs	Platform subscription; free plan sufficient for the prototype

2.4 Literature Survey

Table 2.2 summarises the sources reviewed. They cover the definition and rise of low-code platforms, comparative studies, the practitioners' view of benefits and challenges, and the official Zoho documentation used during development.

Table 2.2: Summary of Literature Survey

S.No.	Author(s) / Source & Year	Title / Product	Key Findings	Relevance to this Project
1	C. Richardson and J. R. Rymer, Forrester Research, 2014 [5]	New Development Platforms Emerge for Customer-Facing Applications	Introduced the term low-code for platforms that speed delivery of business applications through visual development.	Defines the category to which Zoho Creator belongs.
2	A. Sahay et al., Euromicro SEAA, 2020 [6]	Supporting the Understanding and Comparison of Low-Code Development Platforms	Proposes a feature framework to compare platforms by data modelling, UI, logic and deployment support.	Used to organise the features checked in Appendix A.

2407014539

S.No.	Author(s) / Source & Year	Title / Product	Key Findings	Relevance to this Project
3	R. Bock and U. Frank, Business & Information Systems Engineering, 2021 [2]	Low-Code Platform	Explains the concept, benefits and risks such as vendor lock-in and limited customisation.	Basis of the limitations discussed in Section 8.4.
4	D. Di Ruscio et al., Software and Systems Modeling, 2022 [7]	Low-Code Development and Model-Driven Engineering: Two Sides of the Same Coin?	Relates low-code to model-driven engineering and discusses what it adds for citizen developers.	Supports the data-model-first design of Chapter 4.
5	Y. Luo et al., ESEM, 2021 [8]	Characteristics and Challenges of Low-Code Development: The Practitioners' Perspective	Analyses practitioner discussions; reports speed and ease of use as strengths, and platform limits as challenges.	Matches the free-plan limits met during this project.
6	Zoho Corporation [3], [4]	Zoho Creator Help and Deluge Reference	Documents forms, workflows, reports, pages and the Deluge language.	Primary technical reference for the implementation.

2.5 Emerging Trends in Low-Code Development

Three trends are relevant to this application. First, citizen development: platforms increasingly let business users build and change their own applications, which suits a restaurant manager who must update the menu and recipes without a programmer. Second, hyper-automation: routine chains of steps, such as deducting stock when an order is accepted and releasing the table when payment is recorded, are automated through workflows. Third, AI-assisted building: vendors are adding assistants that generate forms and scripts from plain-language prompts. AI assistance was not used in this implementation, but the design is deliberately modular so that such features, for example demand prediction from sales history, could be added later.

2.6 Proposed System

The proposed system is a Zoho Creator application named Restaurant Ordering System. It replaces paper slips with an Order form linked to a table, and replaces handwritten items with Order_Item records that price themselves from the menu. The Kitchen Display Kanban report gives the kitchen and waiters a shared, live view of order status, which removes the status limitation listed in Section 2.2. Recipe records link each dish to its ingredients so that stock is deducted automatically when an order is accepted, and

2407439

a Purchase_Request is raised when an ingredient reaches its reorder level. The Payment form completes the order and frees the table, and the Sales Dashboard presents sales analytics from live data, replacing manual totals.

24070439

CHAPTER 3
REQUIREMENT ANALYSIS

3.1 Stakeholders and User Roles

Table 3.1: Stakeholders and User Roles

Role	Description	Key Responsibilities in the Application
Administrator / Manager	Owner or manager of the restaurant	Maintain menu, categories, recipes, ingredients and tables; review purchase requests; view the Sales Dashboard; manage users
Waiter / Customer	Takes orders at the table or orders for themselves	Create an order for a table; add dishes and quantities; mark orders as served
Kitchen Staff	Prepares the dishes	View the Kitchen Display; accept orders; move orders to Preparing and Ready
Cashier	Handles billing and payment	Record the payment for an order; confirm that the table is released

3.2 Functional Requirements

Table 3.2: Functional Requirements

FR ID	Requirement	Role(s)	Priority
FR-01	The system shall allow the manager to maintain categories, menu items (price, food type, preparation time, availability) and restaurant tables.	Manager	High
FR-02	The system shall maintain ingredients with current stock and reorder level, and recipes linking each dish to its ingredients and quantities.	Manager	High
FR-03	The system shall create an order for a selected table and set the table status to Occupied automatically.	Waiter, Customer	High
FR-04	The system shall add dishes to an order and automatically fetch the dish price and compute the line total from quantity.	Waiter, Customer	High
FR-05	The system shall recalculate the order total whenever an item is added or edited.	System	High
FR-06	The system shall block a dish that is marked unavailable and show an alert.	System	Medium
FR-07	The system shall display orders grouped by status on a Kitchen Display board.	Kitchen Staff, Waiter	High
FR-08	The system shall allow the order status to move through Placed, Accepted, Preparing, Ready, Served, Billed and Paid, with Cancelled as an exit.	Kitchen Staff, Waiter	High
FR-09	The system shall deduct ingredient stock according to the recipe when an order is accepted, and only once per order.	System	High
FR-10	The system shall create a purchase request when an ingredient falls to or below its reorder level.	System	Medium

2407439

FR ID	Requirement	Role(s)	Priority
FR-11	The system shall record a payment, auto-fill the amount from the order total, mark the order Paid and set the table to Free.	Cashier	High
FR-12	The system shall present a dashboard of top-selling items, sales split, order counts by status and revenue trend.	Manager	Medium
FR-13	The system shall keep internal fields (status, total, payment status, stock flag) out of the normal data-entry form.	Administrator	Medium

3.3 Non-Functional Requirements

Table 3.3: Non-Functional Requirements

NFR ID	Category	Requirement
NFR-01	Performance	Reports and the dashboard shall load within 5 seconds for up to 500 order records.
NFR-02	Security	Users shall access only the components permitted by their profile; internal fields shall be hidden from entry forms.
NFR-03	Usability	The application shall be usable on desktop and tablet browsers with a small number of clicks per order.
NFR-04	Availability	The application shall be hosted on the Zoho Creator cloud and available whenever the platform is available.
NFR-05	Maintainability	Business rules shall be kept in short, named workflows so a citizen developer can edit them.
NFR-06	Auditability	Record creation and modification shall be traceable through the platform's record history.

3.4 Hardware and Software Requirements

Table 3.4: Hardware and Software Requirements

Category	Specification
Platform	Zoho Creator, Free plan (trial period completed); data centre: India (zoho.in)
Scripting language	Deluge (Data Enriched Language for the Universal Grid Environment)
Browser	Google Chrome / Microsoft Edge / Mozilla Firefox (latest version)
API testing tool	Not used (no REST integration implemented)
Other Zoho services	None used
Mobile	Zoho Creator mobile app (Android / iOS) supported by the platform; not exercised in this project
Client hardware	Intel Core i3/i5 or above, 8 GB RAM, internet connection

2407459

3.5 Use Case Model

Figure 3.1 shows the actors and the main use cases. Tables 3.5 and 3.6 describe the two most important use cases in detail.

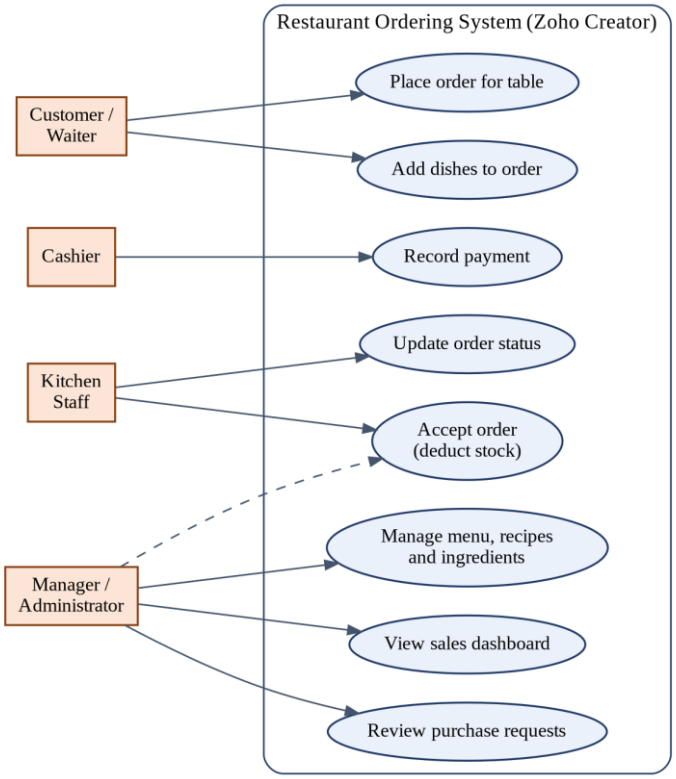


Figure 3.1: Use Case Diagram of the Application

Table 3.5: Use Case Description: Place an Order and Add Dishes

Use Case ID	UC-01
Use Case Name	Place an order and add dishes
Actor(s)	Waiter / Customer
Pre-condition	The menu, categories and tables are set up; the dish is marked Available.
Main Flow	1. The waiter opens the Order form and selects the table. 2. On submit, the table status becomes Occupied and the Order_Item form opens for that order. 3. The waiter selects a dish; the price and line total fill automatically. 4. The waiter sets the quantity; the line total updates. 5. On submit, the order total is recalculated and the form reopens for another dish.
Alternate Flow	If the dish is unavailable, an alert is shown and the item is not saved.
Post-condition	Order_Item records exist for the order and the order Total equals their sum.
Related FR	FR-03, FR-04, FR-05, FR-06

Table 3.6: Use Case Description: Accept an Order and Complete Payment

Use Case ID	UC-02
Use Case Name	Accept an order and complete payment
Actor(s)	Kitchen Staff, Cashier
Pre-condition	An order with at least one item exists with status Placed.
Main Flow	1. Kitchen staff open the Kitchen Display and edit the order, setting Status to Accepted. 2. The system deducts ingredient stock using the recipes and raises a purchase request if stock is low. 3. The order moves through Preparing, Ready and Served. 4. The cashier opens the Payment form and selects the order; the amount is filled automatically. 5. On submit, the order becomes Paid and the table becomes Free.
Alternate Flow	Editing the order again does not deduct stock a second time because Stock_Deducted is set.
Post-condition	Stock reduced once; order Paid; table Free.
Related FR	FR-07, FR-08, FR-09, FR-10, FR-11

2407014 39

CHAPTER 4
SYSTEM DESIGN

4.1 System Architecture

Figure 4.1 shows the layered architecture. Users reach the application through a web browser. The Zoho Creator application layer contains nine forms, nine list reports and a Kanban report, four pivot charts, the Sales Dashboard page and ten form workflows written in Deluge. These read and write the data layer, which is the managed database of the Zoho Creator cloud. No external service is connected in the present version.

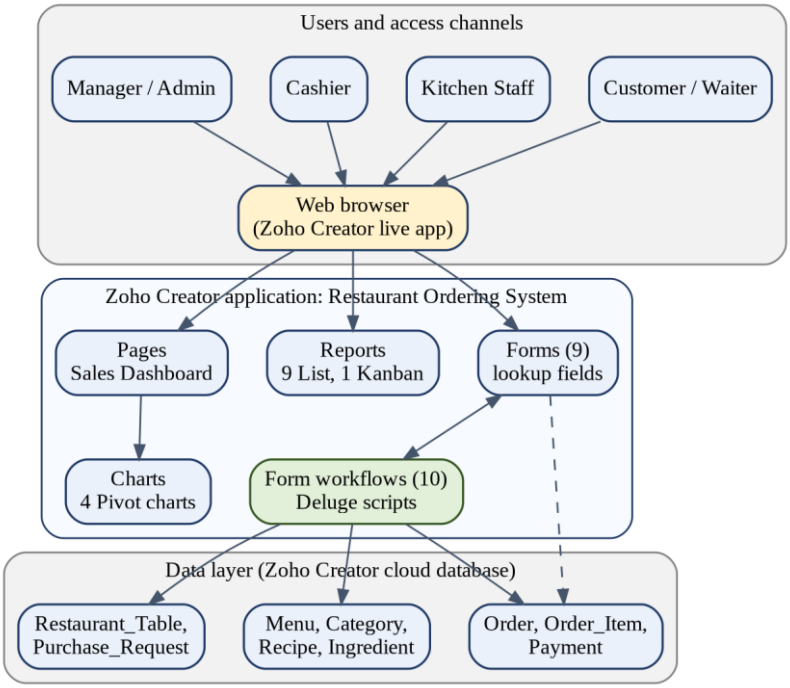


Figure 4.1: System Architecture of the Application

Access layer: staff and customers use a standard browser; the application is responsive, so a tablet in the kitchen is sufficient. **Application layer:** forms capture data, reports present it, the page assembles the dashboard, and workflows react to form events to enforce the business rules. **Data layer:** each form is a table in the Creator database; lookup fields are the foreign keys. **Integration layer:** reserved for future payment, e-mail and WhatsApp integrations.

4.2 Data Model

Relationships in Zoho Creator are realised through lookup fields. The model has three groups: menu and inventory data (Category, Menu_Item, Recipe, Ingredient,

Purchase_Request), floor data (Restaurant_Table) and transaction data (Order, Order_Item, Payment). Figure 4.2 shows the entity-relationship diagram and Table 4.1 lists the relationships. Recipe is a junction form that resolves the many-to-many relationship between dishes and ingredients.

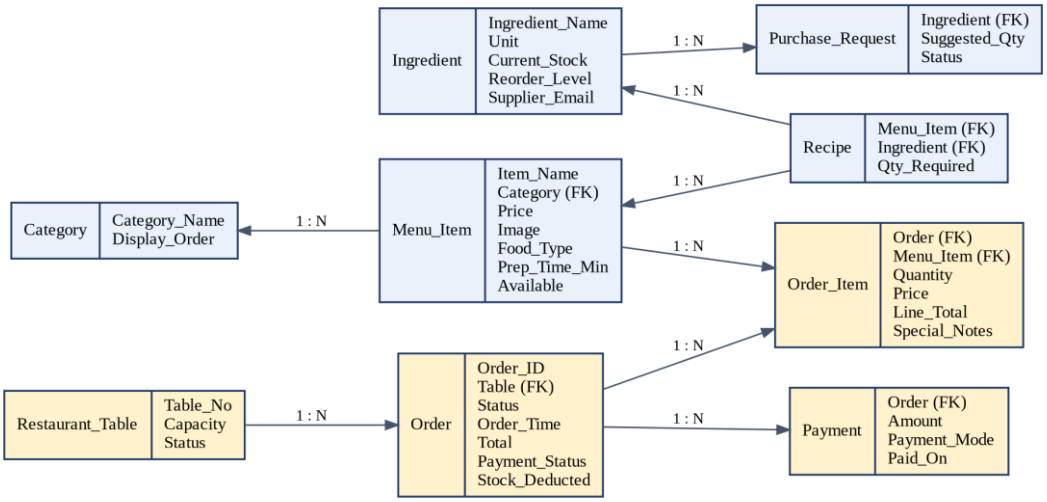


Figure 4.2: Entity-Relationship Diagram

Table 4.1: Entities and Relationships

Entity (Form)	Key Fields	Related To	Relationship Type	Implemented Using
Category	Category_Name, Display_Order	Menu_Item	One-to-Many	Lookup in Menu_Item
Menu_Item	Item_Name, Price, Food_Type, Available	Recipe, Order_Item	One-to-Many	Lookups in Recipe and Order_Item
Ingredient	Ingredient_Name, Current_Stock, Reorder_Level	Recipe, Purchase_Request	One-to-Many	Lookups in Recipe and Purchase_Request
Recipe	Menu_Item, Ingredient, Qty_Required	Menu_Item, Ingredient	Many-to-Many (junction)	Two lookups
Restaurant_Table	Table_No, Capacity, Status	Order	One-to-Many	Lookup in Order
Order	Order_ID, Table, Status, Total, Payment_Status	Order_Item, Payment	One-to-Many (parent-child)	Lookups in Order_Item and Payment
Order_Item	Order, Menu_Item, Quantity, Line_Total	Order, Menu_Item	Many-to-One	Two lookups
Payment	Order, Amount, Payment_Mode, Paid_On	Order	Many-to-One	Lookup

240121

Entity (Form)	Key Fields	Related To	Relationship Type	Implemented Using
Purchase_Request	Ingredient, Suggested_Qty, Status	Ingredient	Many-to-One	Lookup

The parent-child relationship between Order and Order_Item would normally be implemented as a subform. A subform was designed first, but the free plan permits only a demo of subforms, so the same relationship was implemented as a separate Order_Item form linked by a lookup. This master-detail pattern behaves identically for the user and keeps every order line as an independent record that can be reported on directly.

4.3 Form Design

Tables 4.2 to 4.10 give the field specification of each form. Link names are the identifiers used in Deluge scripts. Mandatory is "Yes" only where a field is marked must-have in the form definition.

Table 4.2: Field Specification: Category Form

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Category_Name	Category	Single Line	No	-	Name of the menu category
Display_Order	Display_Order	Number	No	-	Sort position (1 to 3)

Table 4.3: Field Specification: Ingredient Form

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Ingredient_Name	Ingredient_Name	Single Line	No	-	
Unit	Unit	Drop Down	No	kg, g, L, ml, pcs	Choices must match CSV import values
Current_Stock	Current_Stock	Decimal	No	-	Reduced by Deduct Stock workflow
Reorder_Level	Reorder_Level	Decimal	No	-	Threshold for purchase request
Supplier_Email	Supplier_Email	Email	No	-	Flagged as personal data

2401439

Table 4.4: Field Specification: Menu_Item Form

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Item_Name	Item_Name	Single Line	No	-	
Category	Category	Lookup	No	Lookup to Category	Display field: Category_Name
Price	Price	Decimal	No	-	Read by Fetch Price workflow
Image	Image	Image	No	-	Not populated in sample data
Food_Type	Food_Type	Radio	No	Veg, Non-Veg	
Prep_Time_Min	Prep_Time_Min	Number	No	-	Minutes
Available	Available	Checkbox	No	Default: unchecked	Tested by Check Availability

Table 4.5: Field Specification: Restaurant_Table Form

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Table_No	Table_No	Number	No	-	
Capacity	Capacity	Number	No	-	Seats
Status	Status	Drop Down	No	Free, Occupied, Billing	Set by workflows

Table 4.6: Field Specification: Recipe Form

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Menu_Item	Menu_Item	Lookup	No	Lookup to Menu_Item	Display field: Item_Name
Ingredient	Ingredient	Lookup	No	Lookup to Ingredient	Display field: Ingredient_Name
Qty_Required	Qty_Required	Decimal	No	-	Per one portion

Table 4.7: Field Specification: Order Form

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Order_ID	Order_ID	Auto Number	-	Start index 1	System generated
Table	Table	Lookup	No	Lookup to Restaurant_Table	Display field: Table_No
Status	Status	Drop Down	No	Default: Placed	Eight choices; private field
Order_Time	Order_Time	Date-Time	No	Default: current time	Private field
Total	Total	Decimal	No	-	Private; set by workflow

240701439

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Payment_Status	Payment_Status	Drop Down	No	Default: Unpaid	Unpaid, Paid; private
Stock_Deducted	Stock_Deducted	Checkbox	No	Default: unchecked	Guard flag; private

Table 4.8: Field Specification: Order_Item Form

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Order	Order	Lookup	No	Lookup to Order	Display field: Order_ID
Menu_Item	Menu_Item	Lookup	No	Lookup to Menu_Item	Display field: Item_Name
Quantity	Quantity	Number	Yes	Default: 1	
Price	Price	Decimal	No	-	Filled by workflow
Line_Total	Line_Total	Decimal	No	-	Price x Quantity
Special_Notes	Special_Notes	Single Line	No	-	Cooking instructions

Table 4.9: Field Specification: Payment Form

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Order	Order	Lookup	No	Lookup to Order	Display field: Order_ID
Amount	Amount	Number	No	-	Filled by Fetch Amount
Payment_Mode	Payment_Mode	Drop Down	No	Cash, Card, UPI	
Paid_On	Paid_On	Date-Time	No	Default: current time	

Table 4.10: Field Specification: Purchase_Request Form

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Ingredient	Ingredient	Lookup	No	Lookup to Ingredient	Display field: Ingredient_Name
Suggested_Qty	Suggested_Qty	Number	No	-	Twice the reorder level
Status	Status1	Drop Down	No	Pending, Ordered, Received	Link name Status1 because a section already uses Status

4.4 Process and Workflow Design

Figure 4.3 shows the order lifecycle and marks where each of the ten form workflows runs. Green boxes are workflows with Deluge scripts; yellow boxes are user actions; the diamond is the validation that can stop a submission.

2407C1439

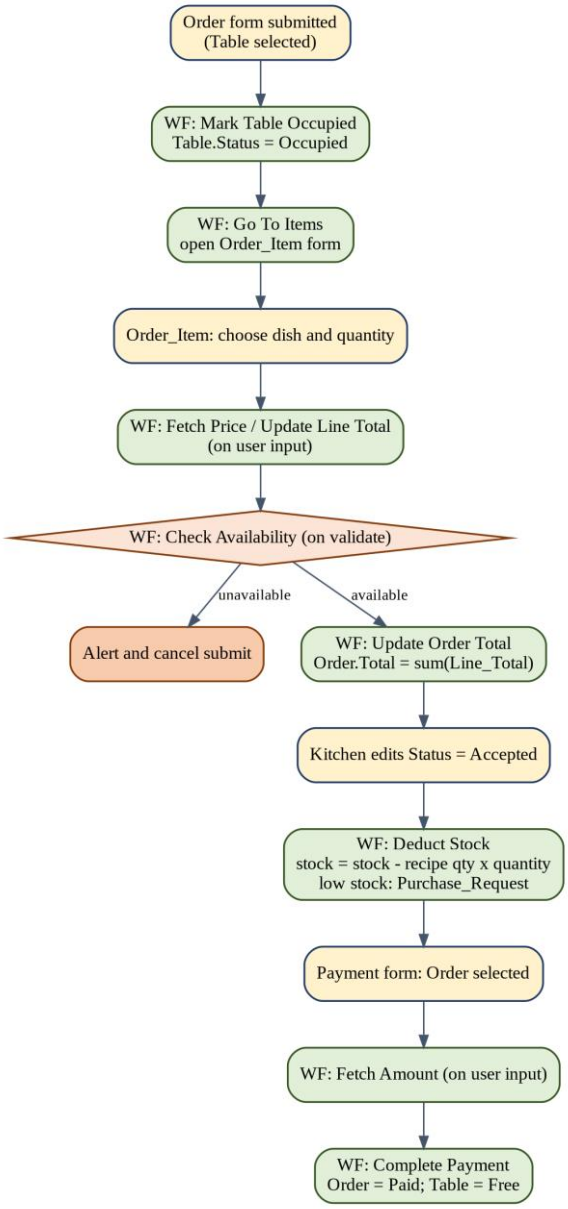


Figure 4.3: Workflow / Process Flow for the Order Lifecycle

4.5 User Interface and Navigation Design

The left navigation lists one section per form, each with its form and report, followed by the Sales Dashboard. The dashboard page combines an HTML snippet with six KPI tiles above a two-by-two grid of the four chart reports. Figure 4.4 shows the navigation structure.

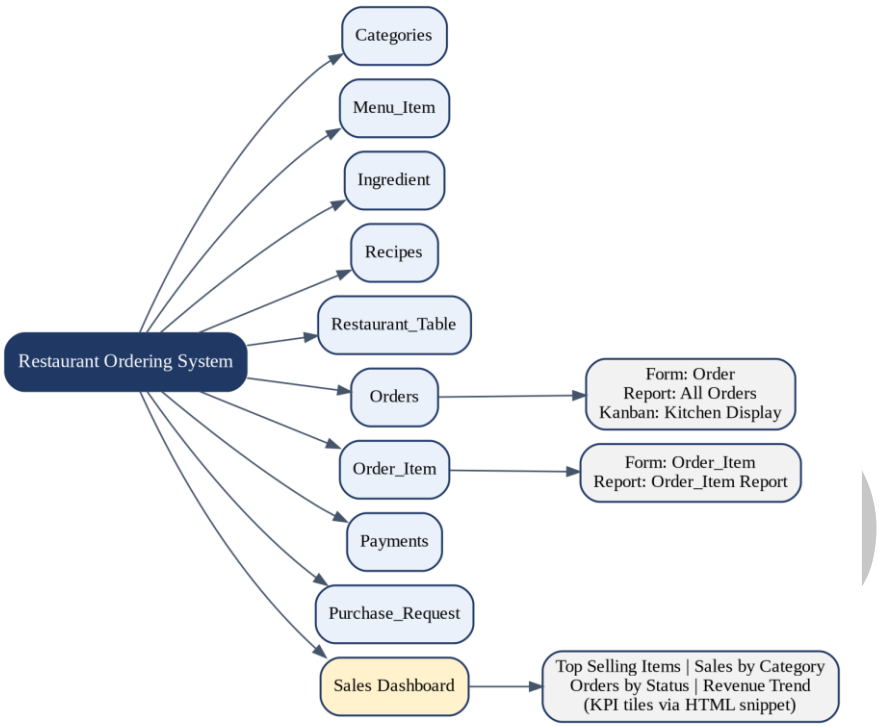


Figure 4.4: Navigation Structure and Wireframe of the Sales Dashboard

4.6 Access Control Design

Table 4.11 defines the intended permission matrix using A = Add, E = Edit, V = View, D = Delete and "-" = no access. The implementation status of this matrix on the free plan is explained in Section 5.8.

Table 4.11: Role-Permission Matrix

Component	Administrator	Waiter	Kitchen	Cashier
Forms: Category, Menu_Item, Ingredient, Recipe, Restaurant_Table	A E V D	V	-	-
Form: Order	A E V D	A V	E V	V
Form: Order_Item	A E V D	A V	V	V
Form: Payment	A E V D	-	-	A V
Form: Purchase_Request	A E V D	-	-	-
Report: Kitchen Display (Kanban)	V	V	E V	-
Report: All Orders	V	V	V	V
Page: Sales Dashboard	V	-	-	-

CHAPTER 5
IMPLEMENTATION

5.1 Application Setup

The application was created from scratch in the Zoho Creator workspace of the student account (India data centre) with the name Restaurant Ordering System and the link name restaurant-ordering-system. The date format is dd-MMM-yyyy, the time zone is Asia/Kolkata and the time format is 24-hour. The app icon shows the text "ROS" on the theme colour. The left menu was organised into one section per form (Categories, Menu_Item, Ingredient, Recipes, Restaurant_Table, Payments, Orders, Purchase_Request, Order_Item) followed by the Sales Dashboard.

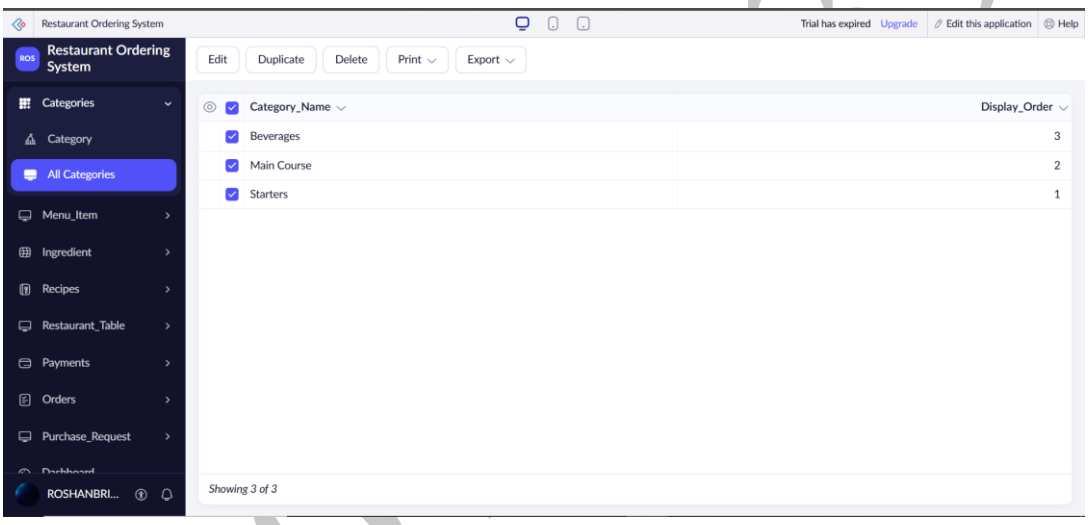


Figure 5.1: Application Home and Navigation Sections

5.2 Forms and Data Capture

Nine forms were built, using eleven distinct field types: Single Line, Number, Decimal, Email, Drop Down, Radio, Checkbox, Image, Lookup, Auto Number and Date-Time. Lookup fields connect the forms (Table 4.1); the lookup display field was set to the human-readable field, for example Item_Name for dishes and Order_ID for orders. Order_ID is an auto-number field. The internal fields of the Order form (Status, Order_Time, Total, Payment_Status and Stock_Deducted) were marked private so that they do not appear on the data-entry form, and defaults were set (Status = Placed, Payment_Status = Unpaid, Order_Time = current time). Quantity in Order_Item is mandatory with a default of 1. Success messages were configured for each form.

Sample data was loaded through CSV import: 3 categories, 10 ingredients, 10 menu items, 22 recipe rows and 5 tables. Lookup columns were matched by name (for

example Item_Name) rather than by ID, and drop-down values in the file had to match the form choices exactly; mismatches produced the import errors recorded in Table 6.2.

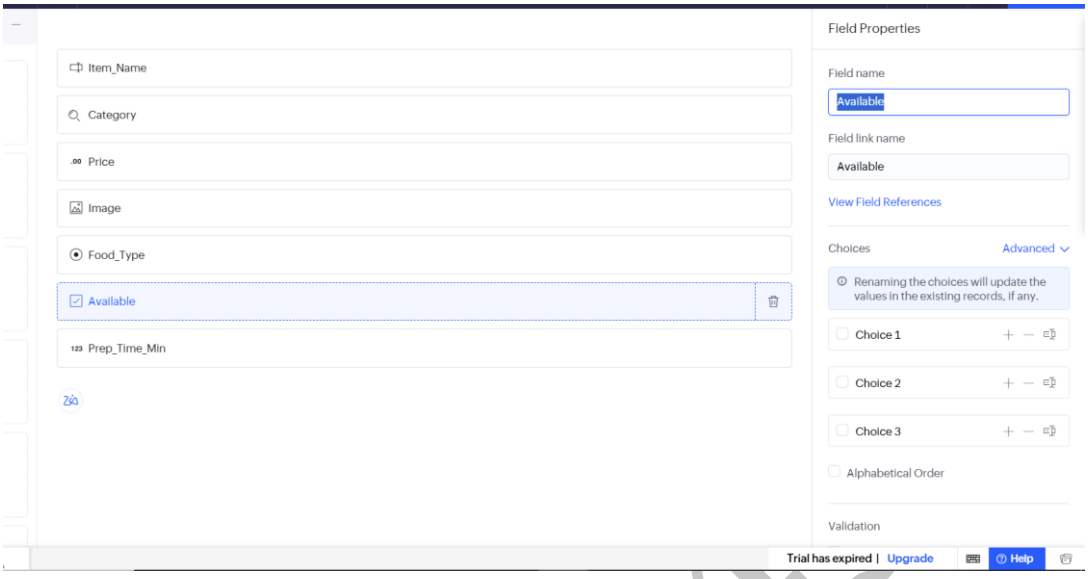


Figure 5.2: Menu_Item Form in the Form Builder

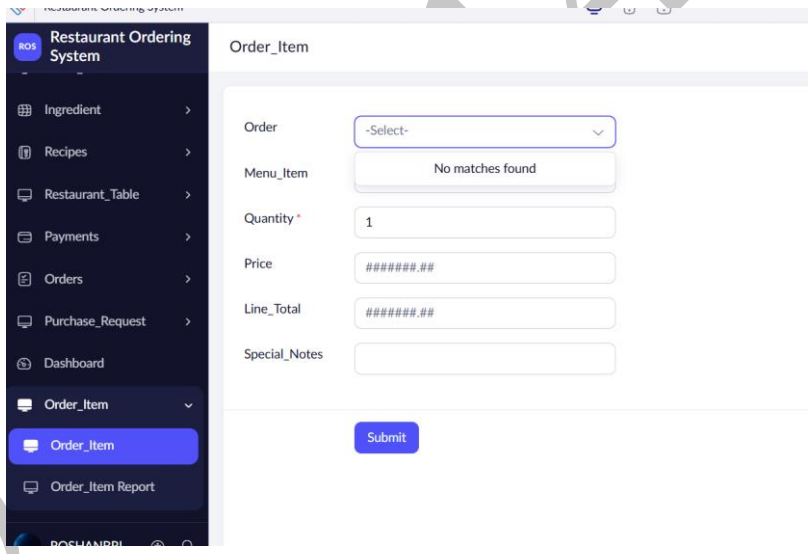


Figure 5.3: Order_Item Form in Live Mode

5.3 Reports

Nine list reports (one per form) were created, together with the Kitchen Display Kanban report on the Order form, which groups order cards into columns by Status, and four pivot-chart reports. Table 5.1 lists them. The All Orders report shows order, table, status, total, payment status and time.

Table 5.1: Reports Implemented

Report Name	Type	Source Form	Filter / Grouping / Sorting	Custom Actions	Roles
All Categories	List	Category	All rows	None	Admin

Report Name	Type	Source Form	Filter / Grouping / Sorting	Custom Actions	Roles
Menu_Item Report	List	Menu_Item	All rows	None	Admin
Ingredient Report	List	Ingredient	All rows	None	Admin
All Recipes	List	Recipe	All rows	None	Admin
Restaurant_Table Report	List	Restaurant_Table	All rows	None	Admin
All Payments	List	Payment	All rows	None	Admin
All Orders	List	Order	All rows; sortable columns	None	Admin
Purchase_Request Report	List	Purchase_Request	All rows	None	Admin
Order_Item Report	List	Order_Item	All rows	None	Admin
Kitchen Display	Kanban	Order	Cards grouped by Status	None	Kitchen, Waiter
Top Selling Items	Pivot chart	Order_Item	Menu_Item vs Sum of Quantity	-	Manager
Sales by Category	Pivot chart	Order_Item	Menu_Item vs Sum of Line_Total (pie)	-	Manager
Orders by Status	Pivot chart	Order	Status vs Count of orders	-	Manager
Revenue Trend	Pivot chart	Order	Order_Time vs Sum of Total (line)	-	Manager

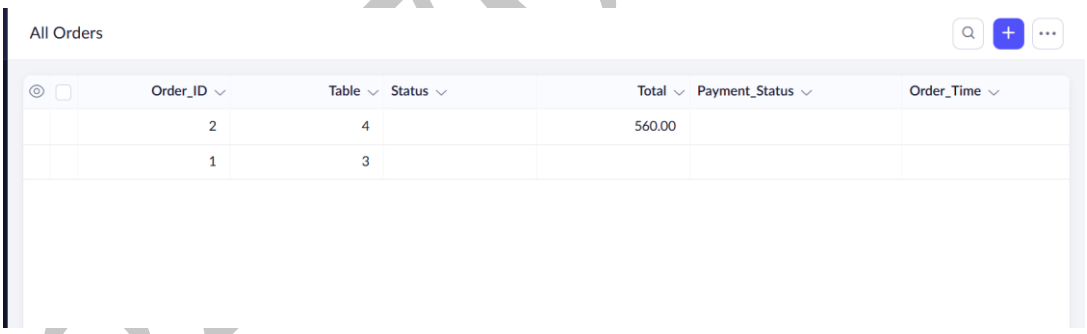


Figure 5.4: All Orders Report

5.4 Pages and Dashboards

The Sales Dashboard page was built in the page builder. A row of six KPI tiles (revenue from paid orders, total orders, active orders, items sold, average order value and low-stock alerts) is produced by an HTML snippet whose Deluge block aggregates the Order, Order_Item and Ingredient forms. Below it, the four pivot charts are embedded as report elements in a two-by-two grid: Top Selling Items (bar), Sales by Category (pie), Orders by Status (bar) and Revenue Trend (line). Because the elements read the live reports, the dashboard updates as soon as an order is paid.

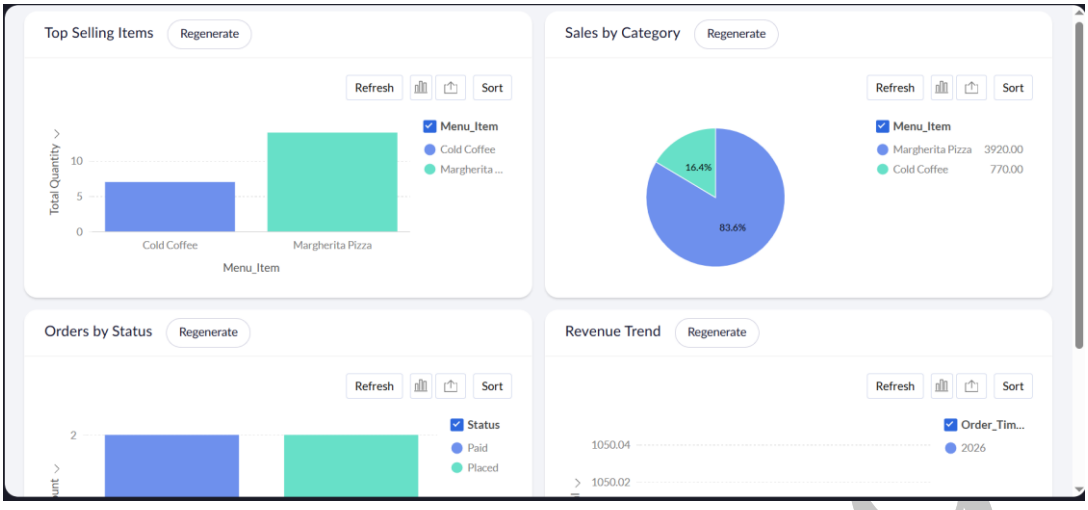


Figure 5.5: Sales Dashboard Page

5.5 Deluge Scripting

Every automation is a short Deluge script attached to a form event. Table 5.2 summarises them; the key scripts are shown below and the complete code is in Appendix C. Scripts use the fetch-by-criteria syntax (for example Order_Item[Order == input.ID]), for each loops, string concatenation and the alert and cancel submit statements.

Table 5.2: Deluge Scripts Implemented

Script ID	Location / Trigger	Purpose	Deluge Concepts Used
DS-01	Order, On Add, On Success	Mark the selected table Occupied	fetch by ID, field update
DS-02	Order, On Add, On Success	Open Order_Item form for the new order	openUrl
DS-03	Order_Item, On Add, On Success	Reopen Order_Item form to add another dish	openUrl
DS-04	Order_Item, User input of Menu_Item	Fetch price and compute line total	fetch by ID, arithmetic
DS-05	Order_Item, User input of Quantity	Recompute line total	arithmetic
DS-06	Order_Item, On Success	Sum line totals into Order.Total	criteria fetch, for each, update
DS-07	Order_Item, On Validate	Block unavailable dishes	fetch, if, alert, cancel submit
DS-08	Order, On Edit, On Success	Deduct stock per recipe; raise purchase request	nested for each, insert into, flag
DS-09	Payment, User input of Order	Auto-fill payment amount	fetch by ID
DS-10	Payment, On Add, On Success	Mark order Paid and free the table	fetch, field update

240701439

Script ID	Location / Trigger	Purpose	Deluge Concepts Used
DS-11	Sales Dashboard, HTML snippet	Compute KPI tiles	aggregate functions, loop

Code 5.1: Price lookup script (DS-04)

```
// DS-04 | Form: Order_Item | Event: User input of Menu_Item
input.Price = Menu_Item[ID == input.Menu_Item].Price;
input.Line_Total = input.Price * input.Quantity;
```

Code 5.2: Availability validation script (DS-07)

```
// DS-07 | Form: Order_Item | Event: On Validate
item = Menu_Item[ID == input.Menu_Item];
if(item.Available == false)
{
    alert item.Item_Name + " is currently unavailable";
    cancel submit;
}
```

Code 5.3: Stock deduction script (DS-08)

```
// DS-08 | Form: Order | Event: On Edit, On Success
if(input.Status == "Accepted" && input.Stock_Deducted == false)
{
    items = Order_Item[Order == input.ID];
    for each i in items
    {
        recipeList = Recipe[Menu_Item == i.Menu_Item];
        for each r in recipeList
        {
            ing = Ingredient[ID == r.Ingredient];
            ing.Current_Stock = ing.Current_Stock - (r.Qty_Required
* i.Quantity);
            if(ing.Current_Stock <= ing.Reorder_Level)
            {
                insert into Purchase_Request
                [
                    Ingredient = ing.ID
                    Suggested_Qty = ing.Reorder_Level * 2
                    Status = "Pending"
                    Added_User = zoho.loginuser
                ];
            }
        }
    }
    o = Order[ID == input.ID];
    o.Stock_Deducted = true;
}
```

DS-04 reads the price from the dish record so that the price is never typed by the waiter. DS-07 runs before the record is saved and stops the submission with an alert if the dish is not available. DS-08 is the core business rule: it loops over the items of the accepted

order, then over the recipe rows of each dish, and subtracts the recipe quantity multiplied by the ordered quantity from the ingredient stock; if the new stock is at or below the reorder level a Purchase_Request is inserted. The Stock_Deducted flag is set at the end so that editing the order later does not deduct the stock again.

5.6 Workflows and Automation

All ten workflows are form workflows. Each has a trigger (record event and form event), an optional condition inside the script, and a Deluge action. No schedule, approval, Blueprint or batch workflow was built: scheduled workflows and Blueprint are restricted on the free plan, and the order lifecycle was handled with a Status drop-down and an edit form instead.

Table 5.3: Workflows Implemented

Workflow Name	Type	Trigger / Event	Condition	Action(s)	Script Ref.
Mark Table Occupied	Form	Order: On Add, On Success	Always	Set table Occupied	DS-01
Go To Items	Form	Order: On Add, On Success	Always	Open Order_Item form	DS-02
Add Another	Form	Order_Item: On Add, On Success	Always	Reopen item form	DS-03
Fetch Price	Form	Order_Item: On Add or Edit, User input of Menu_Item	Always	Fill price and line total	DS-04
Update Line Total	Form	Order_Item: On Add or Edit, User input of Quantity	Always	Recompute line total	DS-05
Update Order Total	Form	Order_Item: On Add or Edit, On Success	Always	Update Order.Total	DS-06
Check Availability	Form	Order_Item: On Add or Edit, On Validate	Dish not available	Alert; cancel submit	DS-07
Deduct Stock	Form	Order: On Edit, On Success	Status = Accepted and not yet deducted	Deduct stock; insert purchase request	DS-08
Fetch Amount	Form	Payment: On Add or Edit, User input of Order	Always	Fill amount	DS-09
Complete Payment	Form	Payment: On Add, On Success	Always	Order Paid; table Free	DS-10

2407139

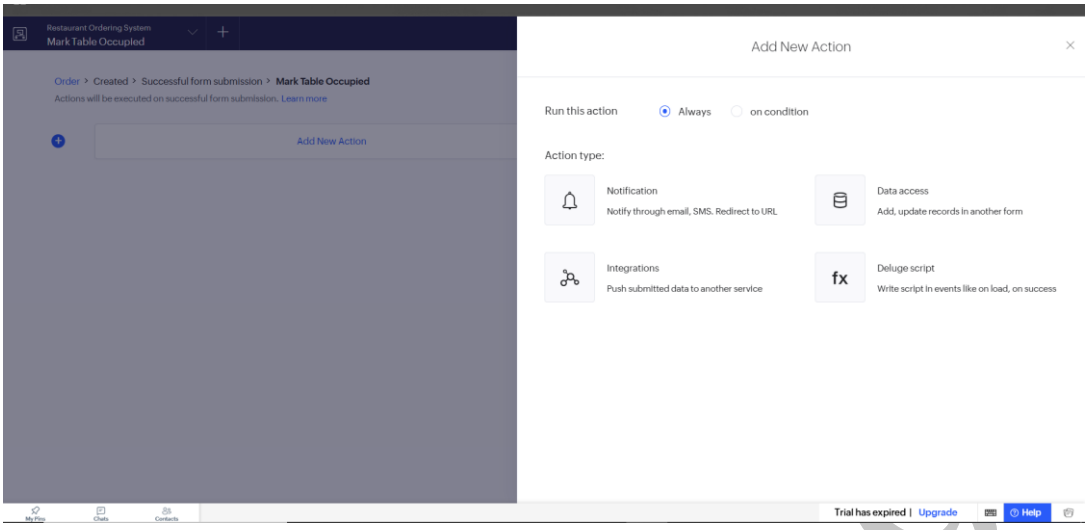


Figure 5.6: Workflow Configuration (Mark Table Occupied, Add New Action)

5.7 Integrations

No external integration was implemented in this version. E-mail notifications to suppliers (sendmail), payment gateway calls and WhatsApp reminders were considered, but outbound e-mail and connection-based integrations are restricted on the free plan after the trial period, and they were not required by the core objectives. The design leaves an obvious extension point: the Deduct Stock script already identifies the low-stock ingredient and its supplier e-mail, so a sendmail statement or an invokeurl call could be added in the same place.

Table 5.4: Integrations Implemented

Integration	Service / Endpoint	Method	Authentication	Data Exchanged / Purpose
Supplier alert	Zoho Mail (sendmail task)	Deluge sendmail	Account sender	Low-stock e-mail to supplier. Not enabled.
Online payment	Razorpay / Stripe	invokeurl or payment field	API key / OAuth	Collect card or UPI payment. Not implemented.
External API	None	None	None	No external API call in this version.

Code 5.4: Planned supplier notification (reference only)

```
// Extension point (not enabled): supplier e-mail on low stock
sendmail
[
    from :zoho.adminuserid
    to :ing.Supplier_Email
    subject :"Low stock alert: " + ing.Ingredient_Name
    message :"Please restock " + ing.Ingredient_Name
]
```

2407143

5.8 User Management and Role-Based Access

The matrix in Table 4.11 is the intended design. In the exported application the standard profiles Administrator, Write, Read, Developer and Customer are present, together with the default role CEO. The free plan limits creation of additional custom profiles and portal users, so the project was developed and tested under the Administrator profile. The internal fields of the Order form are private (hidden from data entry) and the dashboard is a separate menu item. Custom profiles for Waiter, Kitchen and Cashier, and record-level criteria such as Added User == zoho.loginuser, are recorded as future work in Section 9.2.

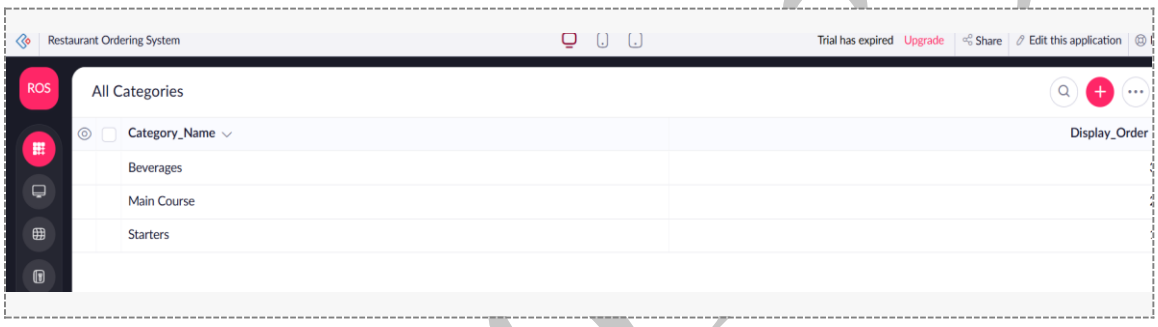


Figure 5.7: Roles and Permission Profiles Configuration

5.9 AI and Analytics Features

No Zia or AI-generated feature was used; all forms, scripts and reports were written and configured by the student, and the scripts were verified by running them against the sample data. For analytics, four pivot-chart reports were built by dragging the Menu_Item, Status or Order_Time column to the X-axis and a numeric column to the Y-axis with an aggregation (Sum or Count), and switching the chart type to bar, pie or line. The Sales Dashboard embeds these charts and adds KPI tiles computed by Deluge aggregate functions. Zoho Analytics integration was not required.

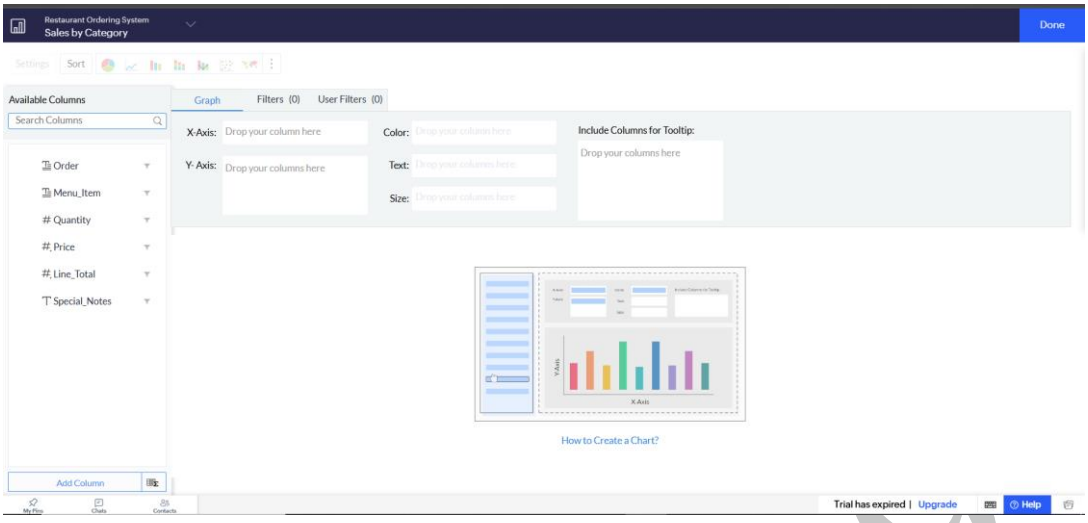


Figure 5.8: Pivot Chart Builder for Sales by Category

5.10 Pro-Code Extension

No pro-code extension (Zoho Catalyst, JavaScript widget or custom Creator REST API) was used, because every requirement could be met with forms, workflows and Deluge. The only code beyond Deluge is the HTML and CSS inside the dashboard snippet. A pro-code extension would become justified if the restaurant needed a branded public ordering website, a payment gateway with server-side verification, or a forecasting model for ingredient demand. These could be built as a Catalyst function called from Deluge with invokeurl, or as a Creator REST API consumed by an external front end.

5.11 Mobile and Portal Access

The application is browser-based and responsive, and the Zoho Creator mobile app can open the same application with the same login. A customer portal (and a QR-code ordering page published with a public permalink) was designed but not enabled, because portal and public publishing are not available on the free plan. The mobile and portal views were therefore not captured in this version.

2407014

CHAPTER 6
TESTING AND DEBUGGING

6.1 Testing Strategy

Testing was done at four levels. Unit testing checked each form on its own: field types, defaults, mandatory fields and success messages. Script testing exercised each Deluge workflow with sample data and compared the result with the expected value (for example, 2 Margherita Pizzas at Rs. 280 must give a line total of Rs. 560). Integration testing followed one order end to end: order, items, acceptance, payment and dashboard. Because the free plan allowed only the Administrator profile, role-based access testing was limited to confirming that internal fields stay hidden from the entry form. Negative tests covered an unavailable dish and a missing quantity.

6.2 Test Cases

Table 6.1 lists the test cases and their traceability to the functional requirements of Table 3.2. Test cases that were executed and observed during development are marked Pass. Cases that must still be executed before submission are marked in yellow so that the actual result can be entered after the live demonstration.

Table 6.1: Test Cases

TC ID	FR Ref.	Test Description	Input / Steps	Expected Result	Actual Result	Status
TC-01	FR-01	Add a menu item with all fields	Fill Menu_Item form and submit	Record created; success message	As expected	Pass
TC-02	FR-01	Import 10 dishes from CSV	Import with Category matched by name	10 records, category shown	As expected (after fix, see BUG-02)	Pass
TC-03	FR-03	Create order for a table	Order form: select Table 3, submit	Order saved; Table 3 becomes Occupied	As expected	Pass
TC-04	FR-04	Price auto-fetch	Order_Item: choose Margherita Pizza	Price = 280	As expected	Pass
TC-05	FR-04	Line total on quantity change	Set Quantity = 2	Line total = 560	As expected	Pass
TC-06	FR-05	Order total on item add	Submit the item	Order Total = 560.00	As expected	Pass
TC-07	FR-05	Order total with several items	Add Cold Coffee x 1 to an order	Total = sum of line totals	As expected	Pass
TC-08	FR-06	Unavailable dish blocked (negative)	Select a dish with Available unchecked	Alert shown; item not saved	As expected	Pass
TC-09	FR-07	Kitchen Display shows order	Open Kitchen Display after creating an order	Card under Placed	[To be verified]	[To be verified]

24U70439

TC ID	FR Ref.	Test Description	Input / Steps	Expected Result	Actual Result	Status
TC-10	FR-08	Status progression	Edit order: Accepted, then Preparing, Ready, Served	Status saved each time	[To be verified]	[To be verified]
TC-11	FR-09	Stock deduction	Accept order of 2 Margherita Pizzas	Cheese 2.0 to 1.8; Flour 15 to 14.6	[To be verified]	[To be verified]
TC-12	FR-09	No double deduction (negative)	Edit the accepted order again	Stock unchanged	[To be verified]	[To be verified]
TC-13	FR-10	Purchase request on low stock	Accept orders until Cheese <= 1	Purchase_Request created	[To be verified]	[To be verified]
TC-14	FR-11	Payment completion	Payment form: select order, submit	Amount auto-filled; order Paid; table Free	As expected	Pass
TC-15	FR-12	Dashboard shows data	Open Sales Dashboard after paid orders	Four charts display current data	As expected	Pass
TC-16	FR-13	Internal fields hidden	Open Order form	Status, Total, Payment_Status not on entry form	As expected	Pass
TC-17	FR-04	Missing quantity (negative)	Clear Quantity and submit	Mandatory-field error	[To be verified]	[To be verified]

6.3 Debugging Deluge Scripts

Errors were found in three ways. The Deluge editor validates syntax and variable names when Save is pressed and displays an error banner naming the line, as in Figure 6.1. For runtime behaviour, values were inspected by running the form and comparing the resulting record with the expected values. The delete-field dependency list (Section 7.4) was used to find which workflows referenced a field before changing it.

A representative bug was the error "Variable 'row' is not defined". The order-line script had been attached to the Price field of the Menu_Item form, but the variable row exists only inside a subform event. Figure 6.1 shows the editor with the error on line 1. The fix was to move the script to the correct form and, later, to use input instead of row when the subform was replaced by the Order_Item form.

2407139

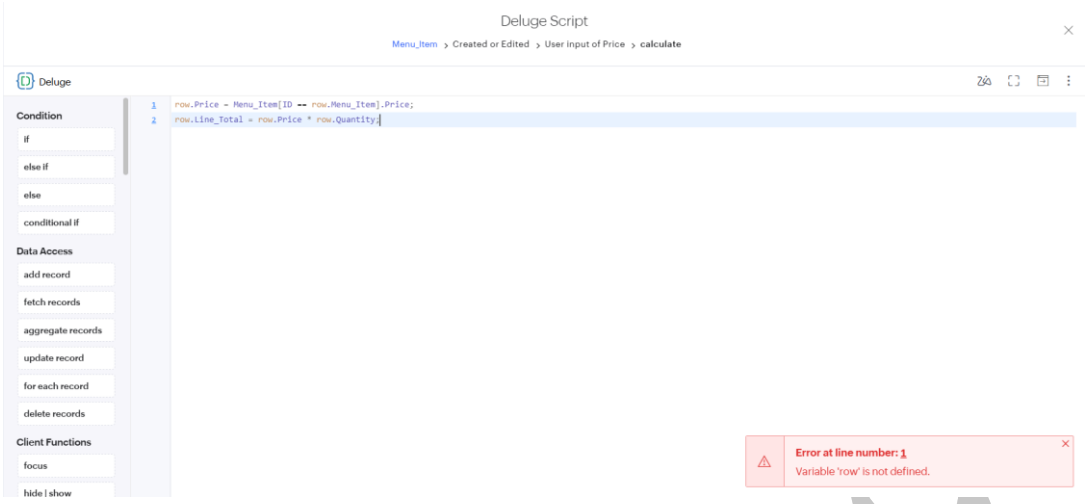


Figure 6.1: Deluge Debug Output: Variable 'row' Is Not Defined

6.4 Defect Log

Table 6.2: Defect Log

Bug ID	Description	Root Cause	Fix Applied	Status
BUG-01	Seven ingredient rows rejected during CSV import	Rows with unit "kg" did not match the Unit drop-down choices	Corrected drop-down choices to kg, g, L, ml, pcs and re-imported the failed rows	Closed
BUG-02	Recipe import filled Menu_Item and Ingredient with invalid values	Lookup columns were matched by record ID instead of by name	Changed matching to Item_Name and Ingredient_Name; ensured parent data existed first	Closed
BUG-03	Menu_Item import failed on the Available column	Checkbox field choices did not accept the CSV text "true"	Imported without that column and set Available on the records	Closed
BUG-04	Variable 'row' is not defined	Script attached to the wrong form; row exists only in a subform event	Moved script to the correct form	Closed
BUG-05	Variable 'Order_Items' is not defined	Subform still had the default name SubForm	Renamed the subform link name	Closed
BUG-06	Plan allows only a demo of the SubForm	Free plan restriction on subforms	Redesigned with a separate Order_Item form linked by lookup	Closed
BUG-07	Variable Available does not exist in Menu_Item	Field was recreated with a different link name	Restored the link name Available and updated the script	Closed
BUG-08	Status and Payment_Status blank in the All Orders report	Defaults were added after the first test orders were created	Set defaults on the fields and corrected the old records	Closed
BUG-09	Subform field could not be deleted	Two old workflows still referenced it	Deleted the old workflows, then removed the subform	Closed

2407439

6.5 User Acceptance Testing

User acceptance feedback must come from at least two people who used the application in an assigned role. The rows below are left for the student to complete after the trial runs.

Table 6.3: User Acceptance Feedback

User	Role Tested	Feedback	Action Taken
[Name 1]	[Waiter]	[ ]	[ ]
[Name 2]	[Kitchen / Cashier]	[ ]	[ ]

24070439

CHAPTER 7
DEPLOYMENT, SECURITY AND GOVERNANCE

7.1 Environments and Deployment Strategy

Zoho Creator offers Development, Stage and Production environments on paid plans. On the free plan used here the application runs in a single environment, so development and testing were done in the same application that is accessed through the live-app URL. To compensate, two controls were used: sample data was kept clearly identifiable, and the application definition was exported as a .ds file on 20 October 2026 and again on 25 October 2026. These exports act as versioned backups and as the means of moving the application to another account, where it can be imported and, on a paid plan, promoted through Stage to Production.

Table 7.1: Deployment Checklist

S.No.	Checklist Item	Done
1	All test cases executed in Development / Stage	Partly (Section 6.2)
2	Sample/test data removed or isolated	Retained for demo
3	Roles, permissions and users configured in Production	N/A (free plan)
4	Connections authorised in Production	N/A (none used)
5	Schedules and workflows enabled	Workflows enabled
6	Application published and URL verified	Yes (live app)
7	Version / release notes recorded	Yes (.ds exports)

7.2 Performance Optimisation

Performance was handled in the design of the scripts rather than measured with a timer, because the data volume of the prototype is small. All data access uses criteria so that only the needed records are read, for example Order_Item[Order == input.ID] reads only the items of one order instead of the whole form. Recipe rows are read once per dish rather than once per ingredient unit, and the stock-deduction script runs only when Status is Accepted and the Stock_Deducted flag is false, so it does nothing on every other edit. Report columns were limited to those needed, and the dashboard reads aggregated pivot charts.

2407439

Table 7.2: Performance Optimisation Results

Optimisation Applied	Component	Before	After	Improvement
Criteria fetch for items of one order	DS-06, DS-08	Read all items and filter in a loop	Read only matching records	Fewer records read
Guard condition before stock loop	DS-08	Loop on every edit	Loop only when Accepted and not yet deducted	No repeated work; no double deduction
Pivot charts instead of scripted totals	Dashboard charts	Totals computed record by record	Platform aggregation	Simpler and faster

No before-and-after timings were recorded; the improvement column is qualitative.

7.3 Security Features

Table 7.3: Security Controls Implemented

Security Control	Where Applied	Threat Mitigated
Private fields	Order: Status, Order_Time, Total, Payment_Status, Stock_Deducted	Accidental or deliberate editing of totals and status at entry
Personal-data flag	Ingredient.Supplier_Email	Exposure of contact details
Validation script	Order_Item On Validate	Invalid transactions for unavailable dishes
Guard flag	Order.Stock_Deducted	Repeated stock deduction
Authenticated access	Whole application (Zoho login)	Unauthorised access
No hard-coded credentials	All scripts	Credential leakage (no connections or keys are used)

7.4 Governance and Administration

Governance practices followed were: consistent link names that are used unchanged in all scripts; descriptive workflow names; regular exports of the application definition; and use of the platform's field-reference check before deleting a field. Figure 7.2 shows that check: when the old subform was deleted, Creator listed the two workflows that still referenced it and blocked the deletion until they were removed, which protects the application from broken scripts. Figure 7.1 shows the Settings page with the permission, personalisation and developer-tool areas used for administration. The student is the administrator and is responsible for maintenance.

2407439

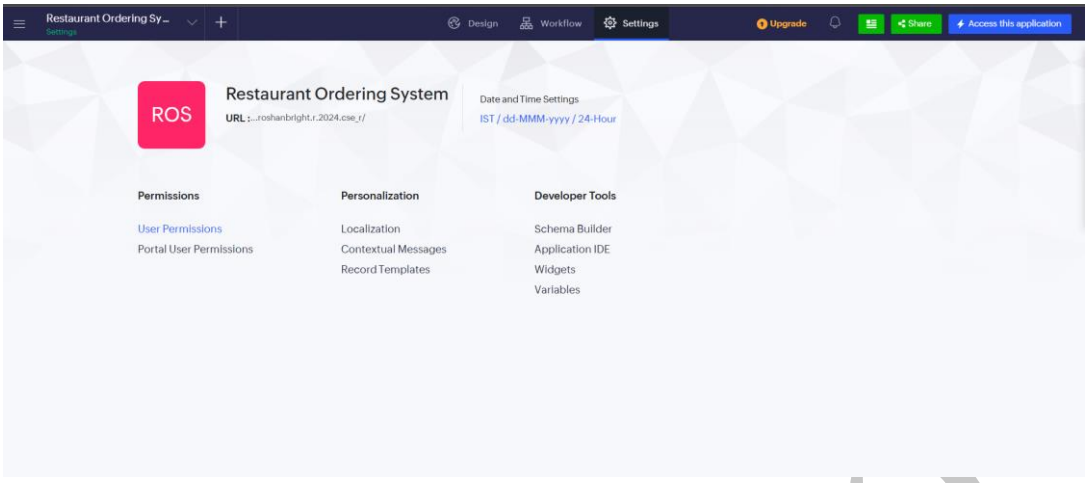


Figure 7.1: Administration Settings

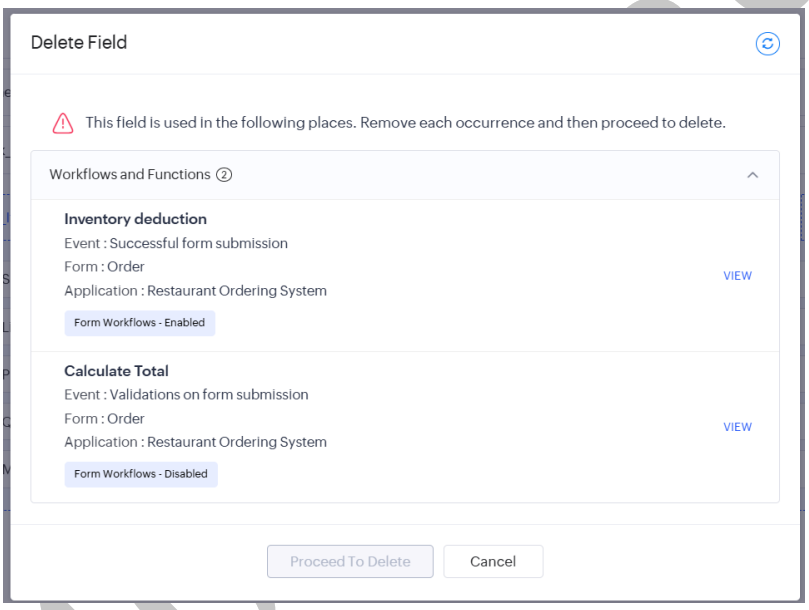


Figure 7.2: Delete Field Dependency Check

CHAPTER 8
RESULTS AND DISCUSSION

8.1 Application Output

The scenario followed is a table of two guests. Step 1: the waiter opens the Order form, selects Table 3 and submits; the table turns Occupied and the Order_Item form opens. Step 2: the waiter adds Margherita Pizza; the price and line total fill automatically, and if a dish is unavailable the alert in Figure 8.2 is shown instead. Step 3: after the order is accepted, prepared, served and paid, the Sales Dashboard in Figure 8.3 reflects the sale.

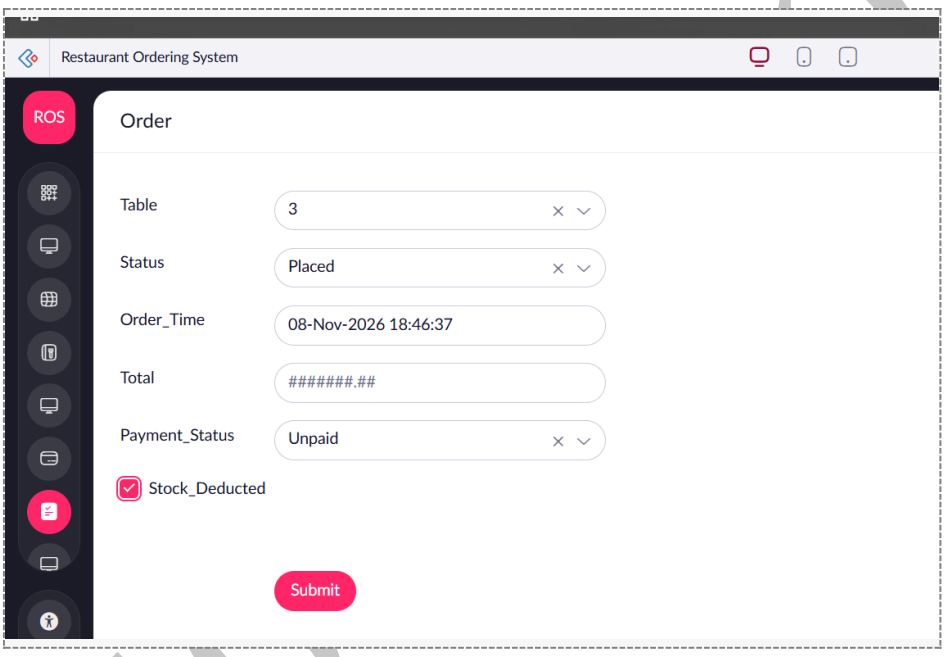


Figure 8.1: Scenario Step 1: Order Placed for a Table

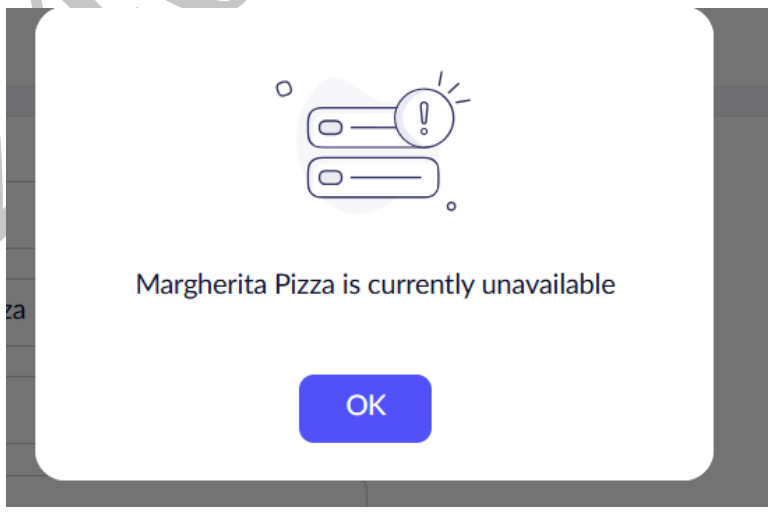


Figure 8.2: Scenario Step 2: Availability Validation on an Order Item

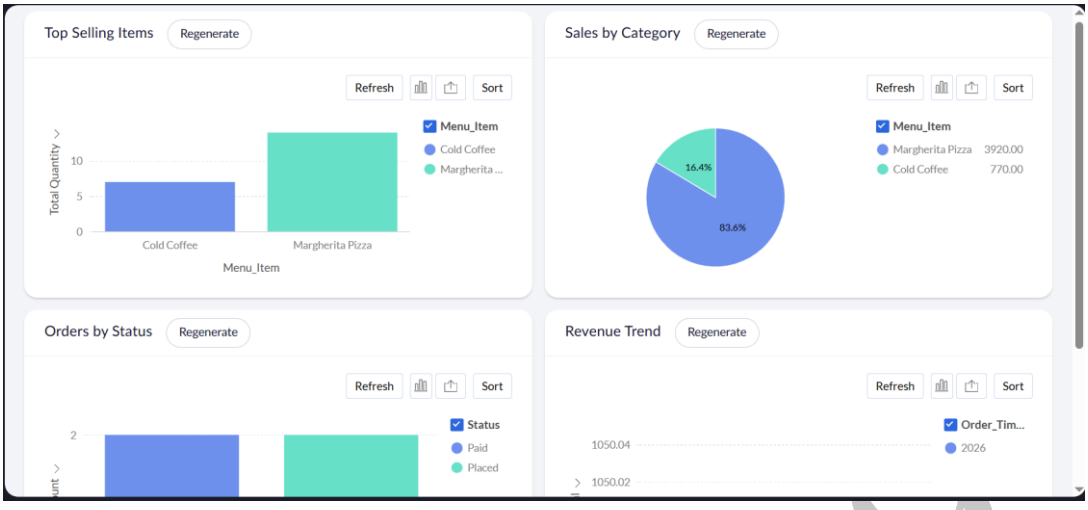


Figure 8.3: Scenario Step 3: Sales Dashboard After Paid Orders

The dashboard in Figure 8.3 shows Cold Coffee and Margherita Pizza as the top-selling dishes, with Margherita Pizza contributing about 83.6% of the sales value in the pie chart, and the Orders by Status chart shows Paid and Placed orders.

8.2 Achievement of Objectives

Table 8.1: Mapping of Objectives to Outcomes

Objective (Sec. 1.3)	Implemented Through	Outcome / Evidence
Normalised data model and forms	Sections 4.2, 4.3, 5.2	Nine forms linked by lookups; Tables 4.1 to 4.10
Automate pricing, totals, table status, stock and payment	Sections 5.5, 5.6	Ten workflows; test cases TC-03 to TC-14
Kitchen display and sales dashboard	Sections 5.3, 5.4	Kanban report and four-chart dashboard; Figure 5.5
Prevent invalid transactions	DS-07, DS-08	Alert on unavailable dish; Stock_Deducted flag
Deliver with governance on the available edition	Sections 7.1 to 7.4	Private fields, .ds backups, dependency checks; limits documented in 8.4

8.3 Comparison with the Existing System

Table 8.2: Existing System versus Proposed Application

Parameter	Existing System	Proposed Application	Improvement
Capturing an order	Handwritten slip carried to kitchen	Order and Order_Item forms; kitchen sees it at once	No lost or misread slips
Price and total	Calculated by hand	Fetched and summed automatically	No arithmetic errors
Order status visibility	Ask the kitchen	Kitchen Display Kanban	Real-time status

240701439

Parameter	Existing System	Proposed Application	Improvement
Stock update	Manual register	Deducted from recipes on acceptance	Accurate stock; early purchase requests
Table release	Informal	Automatic on payment	Faster table turnover
Sales reports	Manual totals	Pivot charts and KPI tiles	Instant analysis

8.4 Limitations of the Low-Code Approach

Several constraints were met. First, the free plan allows only a demo of the subform feature (Figure 8.4), which forced the redesign described in Section 4.2. Second, public permalinks, scheduled workflows, Blueprint and outbound e-mail were not available, so the QR-code customer page, nightly e-mail summaries and the Blueprint status flow were replaced by in-app forms, summary reports and a Status drop-down. Third, CSV import is strict: lookup values must match names exactly and drop-down values must match choices, which caused BUG-01 to BUG-03. Fourth, the dashboard page offers limited layout control, so KPI tiles needed an HTML snippet. Finally, the application depends on the Zoho platform and its plan limits, a form of vendor lock-in noted in the literature [2].

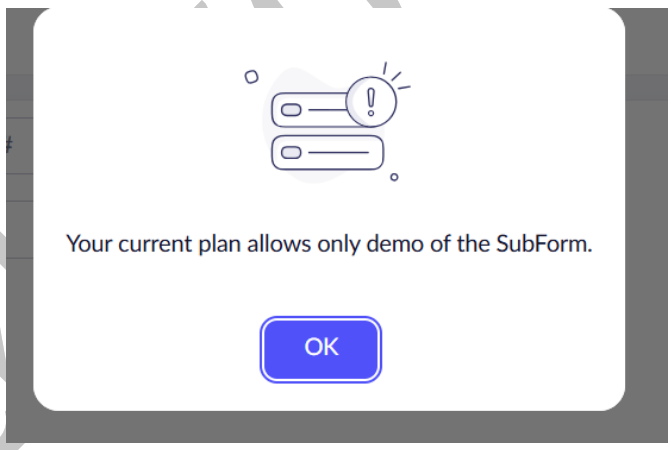


Figure 8.4: Subform Restriction on the Free Plan

8.5 Reflection on the No-Code, Low-Code and Pro-Code Spectrum

Table 8.3: Position of Application Components on the Spectrum

Component	Level	Reason
Forms, fields, lookups, defaults	No-code	Configured in the form builder
List, Kanban and pivot-chart reports	No-code	Configured by dragging columns and choosing options
Pricing, totals, table status, payment completion	Low-code	Short Deluge scripts in form workflows

Component	Level	Reason
Stock deduction and purchase requests	Low-code	Nested loops and criteria fetches in Deluge
Dashboard KPI tiles	Low-code	HTML and CSS snippet with Deluge aggregation
Public QR ordering site, payment gateway, demand forecasting	Pro-code (future)	Would need Catalyst functions, REST APIs or an external front end

The reflection is that about two thirds of the application is no-code or very light low-code. I would move to pro-code only when a requirement cannot be expressed in forms, workflows and Deluge, such as a branded public storefront or machine-learning forecasts.

2407014 39

CHAPTER 9
CONCLUSION AND FUTURE ENHANCEMENTS

9.1 Conclusion

The Restaurant Ordering and Kitchen Display System shows that the complete order-to-payment cycle of a restaurant can be digitised on Zoho Creator without conventional programming. Nine linked forms, ten Deluge-based form workflows, a Kanban kitchen display, four pivot charts and a sales dashboard were built and tested. The application prices orders automatically, tracks them through the kitchen, deducts ingredient stock from recipes exactly once, raises purchase requests, closes the order on payment and frees the table.

The work also documents the practical limits of the free plan: subforms, public publishing, scheduled e-mail and Blueprint were unavailable and were replaced by a master-detail design and in-app flows. These workarounds kept every functional requirement achievable while showing where a paid plan or a pro-code extension would add value.

9.2 Future Enhancements

- Public QR-code ordering page for customers, using a public permalink and a customer portal on a paid plan.
- Online payments through Razorpay or Stripe, with payment confirmation written back to the Payment form.
- Supplier e-mail and WhatsApp alerts from the stock-deduction script, and a scheduled daily sales summary to the owner.
- Custom profiles for Waiter, Kitchen and Cashier with record-level criteria, and a Blueprint for the order lifecycle.
- Customer feedback form with rating analytics, and a Catalyst-based forecast of ingredient demand from sales history.

9.3 Learning Outcomes and Career Relevance

The project developed skills in data modelling with lookups, form and report design, Deluge scripting, workflow automation, debugging and dashboard building, together with an understanding of platform limits and plan restrictions. These map directly to roles such as citizen developer, low-code developer, solutions consultant and

2407439

automation engineer, where the ability to turn a business process into a working application quickly, and to recognise when to extend it with pro-code, is in demand [2], [8].

24070439

REFERENCES

[1] W. Rosam and O. Kalnichevski, *Low-Code Development: A Practical Guide*. New Delhi, India: BPB Publications.

[2] R. Bock and U. Frank, "Low-Code Platform," *Business & Information Systems Engineering*, vol. 63, no. 6, pp. 733-740, 2021.

[3] Zoho Corporation, "Zoho Creator Help Documentation." [Online]. Available: https://www.zoho.com/creator/help.

[4] Zoho Corporation, "Deluge Script Reference Guide." [Online]. Available: https://www.zoho.com/deluge.

[5] C. Richardson and J. R. Rymer, "New Development Platforms Emerge for Customer-Facing Applications," Forrester Research, Cambridge, MA, USA, Jun. 2014.

[6] A. Sahay, A. Indamutsa, D. Di Ruscio, and A. Pierantonio, "Supporting the understanding and comparison of low-code development platforms," in *Proc. 46th Euromicro Conf. Softw. Eng. Adv. Appl. (SEAA)*, 2020, pp. 171-178.

[7] D. Di Ruscio, D. Kolovos, J. de Lara, A. Pierantonio, M. Tisi, and M. Wimmer, "Low-code development and model-driven engineering: Two sides of the same coin?," *Softw. Syst. Model.*, vol. 21, no. 2, pp. 437-446, 2022.

[8] Y. Luo, P. Liang, C. Wang, M. Shahin, and J. Zhan, "Characteristics and challenges of low-code development: The practitioners' perspective," in *Proc. 15th ACM/IEEE Int. Symp. Empir. Softw. Eng. Meas. (ESEM)*, 2021.

24070439

APPENDIX A - ZOHO CREATOR FEATURE COVERAGE CHECKLIST

Each feature is marked as implemented (Y), not implemented (N) or partly implemented (Partial) and linked to the report section that evidences it.

Table A.1: Feature Coverage Checklist

Unit	Zoho Creator Feature	Done	Where in the Application	Report Section	Figure / Code No.
I	Application created & sections/menu organised	Y	Restaurant Ordering System; left menu sections	5.1	Figure 5.1
II	Forms with 8 or more distinct field types	Y	Eleven field types across nine forms	5.2	Tables 4.2 to 4.10
II	Lookup field (relationship between forms)	Y	Menu_Item, Recipe, Order, Order_Item, Payment	4.2, 5.2	Figure 4.2
II	Subform (parent-child data)	N	Free plan allows demo only; replaced by Order_Item form	4.2, 8.4	Figure 8.4
II	Field validations / layout rules	Y	Mandatory Quantity, private fields, On Validate script	5.2, 5.5	Code 5.2
II	List report with filter, grouping, custom action	Y	Nine list reports (no custom actions)	5.3	Figure 5.4
II	Kanban / Calendar / Timeline / Map report	Y	Kitchen Display (Kanban)	5.3	Table 5.1
II	Pivot table / chart report	Y	Four pivot charts	5.3, 5.9	Figure 5.8
II	Page / dashboard built with UI Builder	Y	Sales Dashboard	5.4	Figure 5.5
II	Deluge: form events (validate / success / field actions)	Y	On Validate, On Success, User input	5.5	Code 5.1 to 5.3
II	Deluge: functions, fetch, loops, collections	Y	Fetch by criteria and nested for each (no standalone function)	5.5	Code 5.3
II	Email / in-app notifications	N	Outbound e-mail restricted on free plan; planned	5.7	Code 5.4
II	Form workflow	Y	Ten form workflows	5.6	Table 5.3
II	Scheduled workflow	N	Restricted on free plan	8.4	-
II	Approval process	N	Not required by the process	8.4	-
II	Blueprint	N	Not available on free plan; Status drop-down used	8.4	-
II	Batch / report workflow	N	Not required	8.4	-
II	Testing & debugging evidence	Y	Test cases, defect log, debug screenshot	6	Figure 6.1
III	Integration with a Zoho service	N	None used	5.7	-
III	Connection / REST API call (invokeurl)	N	None used	5.7	-
III	Webhook or Creator REST API	N	None used	5.7	-
III	Users, roles and permission profiles	Partial	Default profiles and role present; custom profiles not created	5.8	-
III	Record-level access criteria / customer portal	N	Portal not available on free plan	5.8	-

2407/1439

Unit	Zoho Creator Feature	Done	Where in the Application	Report Section	Figure / Code No.
III	Zia / AI feature	N	Not used	5.9	-
III	Analytics (charts / Zoho Analytics)	Y	Pivot charts and KPI tiles	5.9	Figure 5.8
IV	Performance optimisation applied & measured	Partial	Applied; not timed	7.2	Table 7.2
IV	Deployment across environments	N	Single environment; .ds exports taken	7.1	Table 7.1
IV	Security controls (encryption, OAuth, permissions)	Y	Private fields, personal-data flag	7.3	Table 7.3
IV	Governance (audit trail, administration)	Partial	Settings and dependency check used	7.4	Figure 7.1
V	Pro-code extension (Catalyst / widgets / functions)	N	Justified in 5.10	5.10	-
V	Low-code limitations & spectrum analysis	Y	Sections 8.4 and 8.5	8.4, 8.5	Table 8.3
-	Mobile app / portal access	N	Not exercised on free plan	5.11	-

Total features implemented: 15 / 32 (plus 3 partly implemented)

24070143

APPENDIX B - COURSE OUTCOME MAPPING

Table B.1: Mapping of Course Outcomes to Report Chapters

CO	Course Outcome Statement	Evidenced In	Self-Assessed Level	Examiner Level
CO1	Explain the principles, architecture, and business value of low-code application development using Zoho Creator.	Ch. 1, 2	[ ]	
CO2	Develop business applications using Zoho Creator forms, reports, workflows, and Deluge scripting.	Ch. 4, 5 (5.1-5.6), 6	[ ]	
CO3	Integrate Zoho Creator applications with Zoho services and external systems using APIs, connections, and role-based security.	Ch. 4.6, 5.7-5.9	[ ]	
CO4	Deploy, secure, and manage Zoho Creator applications across development and production environments following governance best practices.	Ch. 7	[ ]	
CO5	Evaluate low-code development strategies, AI-assisted application development, and the transition from citizen developer to professional developer.	Ch. 5.9, 5.10, 8.4, 8.5, 9.3	[ ]	

2407014439

APPENDIX C - COMPLETE DELUGE SOURCE CODE

C.1 Order form

```
// Script ID : DS-01
// Location  : Order: On Add, On Success (Mark Table Occupied)
// Purpose   : Mark the selected table as Occupied

t = Restaurant_Table[ID == input.Table];
t.Status = "Occupied";
```

```
// Script ID : DS-02
// Location  : Order: On Add, On Success (Go To Items)
// Purpose   : Open the Order_Item form for the new order

openUrl("#Form:Order_Item?Order=" + input.ID, "same window");
```

```
// Script ID : DS-08
// Location  : Order: On Edit, On Success (Deduct Stock)
// Purpose   : Deduct ingredient stock once per accepted order and
raise purchase requests

if(input.Status == "Accepted" && input.Stock_Deducted == false)
{
    items = Order_Item[Order == input.ID];
    for each i in items
    {
        recipeList = Recipe[Menu_Item == i.Menu_Item];
        for each r in recipeList
        {
            ing = Ingredient[ID == r.Ingredient];
            ing.Current_Stock = ing.Current_Stock - (r.Qty_Required
* i.Quantity);
            if(ing.Current_Stock <= ing.Reorder_Level)
            {
                insert into Purchase_Request
                [
                    Ingredient = ing.ID
                    Suggested_Qty = ing.Reorder_Level * 2
                    Status = "Pending"
                    Added_User = zoho.loginuser
                ];
            }
        }
    }
    o = Order[ID == input.ID];
    o.Stock_Deducted = true;
}
```

C.2 Order_Item form

```
// Script ID : DS-03
// Location  : Order_Item: On Add, On Success (Add Another)
// Purpose   : Reopen the item form for the same order
```

```
openUrl("#Form:Order_Item?Order=" + input.Order, "same window");
```

```
// Script ID : DS-04
// Location  : Order_Item: User input of Menu_Item (Fetch Price)
// Purpose   : Fetch dish price and compute line total

input.Price = Menu_Item[ID == input.Menu_Item].Price;
input.Line_Total = input.Price * input.Quantity;
```

```
// Script ID : DS-05
// Location  : Order_Item: User input of Quantity (Update Line
Total)
// Purpose   : Recompute line total

input.Line_Total = input.Price * input.Quantity;
```

```
// Script ID : DS-06
// Location  : Order_Item: On Success (Update Order Total)
// Purpose   : Sum all line totals into Order.Total

items = Order_Item[Order == input.Order];
total = 0;
for each i in items
{
    total = total + i.Line_Total;
}
o = Order[ID == input.Order];
o.Total = total;
```

```
// Script ID : DS-07
// Location  : Order_Item: On Validate (Check Availability)
// Purpose   : Block unavailable dishes

item = Menu_Item[ID == input.Menu_Item];
if(item.Available == false)
{
    alert item.Item_Name + " is currently unavailable";
    cancel submit;
}
```

C.3 Payment form

```
// Script ID : DS-09
// Location  : Payment: User input of Order (Fetch Amount)
// Purpose   : Auto-fill the payment amount

input.Amount = Order[ID == input.Order].Total;
```

```
// Script ID : DS-10
// Location  : Payment: On Add, On Success (Complete Payment)
// Purpose   : Mark the order Paid and free the table

o = Order[ID == input.Order];
o.Payment_Status = "Paid";
o.Status = "Paid";
t = Restaurant_Table[ID == o.Table];
t.Status = "Free";
```

C.4 Sales Dashboard (HTML snippet, Deluge block)

```
// Script ID : DS-11
// Location  : Page: Sales Dashboard, HTML snippet
// Purpose   : Compute the six KPI tiles

paidOrders   = Order[Payment_Status == "Paid"];
paidCount    = paidOrders.count();
revenue      = ifnull(paidOrders.sum(Total), 0);
totalOrders  = Order[ID != 0].count();
activeOrders = Order[Status != "Paid" && Status !=
"Cancelled"].count();
itemsSold    = ifnull(Order_Item[ID != 0].sum(Quantity), 0);
avgOrder     = 0;
if(paidCount > 0)
{
    avgOrder = (revenue / paidCount).round(2);
}
lowCount = 0;
for each ing in Ingredient[ID != 0]
{
    if(ing.Current_Stock <= ing.Reorder_Level)
    {
        lowCount = lowCount + 1;
    }
}
```

APPENDIX D - USER MANUAL

D.1 Accessing the Application

Open the application URL https://creatorapp.zoho.in/roshanbright.r.2024.cse_r/restaurant-ordering-system/ in a browser and sign in with the Zoho account that has been shared with you. On a phone or tablet, install the Zoho Creator app, sign in and open Restaurant Ordering System. The left menu lists Categories, Menu_Item, Ingredient, Recipes, Restaurant_Table, Payments, Orders, Purchase_Request, Order_Item and Sales Dashboard.

D.2 Instructions for the Waiter

1. Open Orders, then the Order form. Select the table and click Submit. The table becomes Occupied.
2. The Order_Item form opens. Choose the dish; the price fills automatically. Set the quantity and click Submit. If a message says the dish is unavailable, choose another dish.
3. Repeat for each dish. The form reopens for the same order; the order Total updates after each dish.
4. When the Kitchen Display shows the order as Ready, serve it and set the status to Served.

D.3 Instructions for the Kitchen Staff

1. Open the Kitchen Display. New orders appear under Placed.
2. Open the order, edit it, set Status to Accepted and submit. Ingredient stock is deducted automatically.
3. Update the status to Preparing and then Ready when the dishes are done.

D.4 Instructions for the Cashier

1. Open Payments, then the Payment form. Select the order; the amount fills automatically.
2. Choose Cash, Card or UPI and click Submit. The order becomes Paid and the table is released.

240701439

D.5 Instructions for the Manager

1. Maintain dishes in Menu_Item (tick Available), ingredients in Ingredient and recipes in Recipes.
2. Review Purchase_Request for ingredients that need to be re-ordered.
3. Open the Sales Dashboard for top sellers, sales split, order status and revenue trend.

24070439

APPENDIX E - EVALUATION RUBRIC

(For Examiner Use Only)

Student Name		**Register No.**	
Project Title	Restaurant Ordering and Kitchen Display System	**Date**	

Table E.1: Evaluation Rubric for the Individual Project

S.No.	Criterion	Indicators	Max. Marks	Marks Awarded
1	Problem Definition & Requirement Analysis	Clarity of problem, measurable objectives, complete FR/NFR, use cases	10	
2	System Design	Architecture, ER/data model, form specifications, workflow design, permission matrix	10	
3	Forms, Reports & Pages	Variety of field types, lookups/subforms, report types, UI Builder dashboards	15	
4	Deluge Scripting & Workflows	Correctness and depth of scripts; form, schedule, approval, Blueprint, batch workflows	15	
5	Integration, RBAC & AI	Zoho/external integrations, connections, roles & permissions, Zia/analytics	10	
6	Testing, Deployment, Security & Governance	Test coverage, environments, optimisation, security controls, audit	10	
7	Live Demonstration	End-to-end working of the application across roles; feature checklist verified	15	
8	Project Report	Adherence to template and format, clarity, figures, references	5	
9	Viva-Voce	Understanding of the platform, design decisions and low-code concepts	10	
	TOTAL		**100**	

Table E.2: Performance Level Descriptors

Level	% of Max. Marks	Descriptor
Excellent	90-100	Feature implemented completely, correctly and beyond the basic requirement; student explains design choices confidently.
Good	70-89	Feature implemented correctly with minor gaps; explanation adequate.
Satisfactory	50-69	Feature partially implemented or works only for the main path; limited explanation.
Needs Improvement	< 50	Feature missing, non-functional or not understood by the student.

240701439

Signature of Internal Examiner Name: _____	**Signature of External Examiner** Name: _____

240701439

APPENDIX F - VIVA-VOCE RECORD SHEET

(For Examiner Use Only)

Student Name	[ ]	**Register No.**	[ ]
Department / Section	[ ]	**Date of Viva**	[ ]
Application URL Verified	Yes / No	**Demo Duration**	_____ min

Table F.1: Viva-Voce Questions and Responses

Q.No.	Question / Topic Asked	Unit	Response (E / G / S / NI)	Remarks
1				
2				
3				
4				
5				
6				
7				
8				

E - Excellent G - Good S - Satisfactory NI - Needs Improvement

Table F.2: Marks Summary

Component	Max. Marks	Marks Awarded
Criteria 1-8 (Appendix E)	90	
Viva-Voce	10	
TOTAL	**100**	

Examiners' Remarks:

_____

_____

_____

_____

INTERNAL EXAMINER Name: _____	**EXTERNAL EXAMINER** Name: _____

2407439